

Entonces, como $2^k = n$, se sabe que $k = \log n$. Como $T(1) \leq c_1$, (9.12) es

$$T(n) \leq c_1 n + c_2 n \log n \quad (9.13)$$

La ecuación (9.13) es en realidad la mejor cota que se pudo colocar en $T(n)$, y demuestra que $T(n)$ es $O(n \log n)$.

9.4 Solución general para una clase grande de recurrencias

Considérese la recurrencia que surge al dividir un problema de tamaño n en a subproblemas de tamaño n/b . Por conveniencia, se supone que un problema de tamaño 1 requiere una unidad de tiempo y que el tiempo para reunir las soluciones de los subproblemas y obtener una solución del problema de tamaño n es $d(n)$, en las mismas unidades de tiempo. Para el ejemplo de *mergesort*, se tienen $a = b = 2$ y $d(n) = c_2 n / c_1$, en unidades de c_1 . Entonces, si $T(n)$ es el tiempo para resolver un problema de tamaño n , se tiene

$$\begin{aligned} T(1) &= 1 \\ T(n) &= aT(n/b) + d(n) \end{aligned} \quad (9.14)$$

Obsérvese que (9.14) sólo se aplica a las n que sean una potencia entera de b , pero si se presume que $T(n)$ es continua, al tomar una cota superior ajustada sobre $T(n)$ para aquellos valores de n , nos indica cómo crece $T(n)$ en general.

Obsérvese también que se utiliza la igualdad en (9.14), mientras que en (9.1) hay desigualdades. La razón es que aquí $d(n)$ puede ser arbitraria y, por tanto, exacta, mientras que en (9.1) la suposición de que $c_2 n$ fue el peor caso en tiempo de combinación, para una constante c_2 y toda n , fue sólo una cota superior; el peor caso real del tiempo de ejecución con entradas de tamaño n pudo haber sido menor que $2T(n/2) + c_2 n$. En realidad, hay muy poca diferencia entre utilizar $=$ o $\leq$ en la recurrencia, ya que de cualquier modo se obtiene una cota superior para el peor caso del tiempo de ejecución.

Para resolver (9.14) se aplica la técnica de sustituciones repetidas para T en el lado derecho, igual que se hizo para un ejemplo específico en la exposición anterior de la expansión de recurrencias. Esto es, la sustitución n/b^i por n en la segunda línea de (9.14) da

$$T\left(\frac{n}{b^i}\right) = aT\left(\frac{n}{b^{i+1}}\right) + d\left(\frac{n}{b^i}\right) \quad (9.15)$$

Así, al empezar con (9.14) y sustituir (9.15) para $i = 1, 2, \dots$, se tiene

$$\begin{aligned} T(n) &= aT\left(\frac{n}{b}\right) + d(n) \\ &= a\left[aT\left(\frac{n}{b^2}\right) + d\left(\frac{n}{b}\right)\right] + d(n) = a^2T\left(\frac{n}{b^2}\right) + ad\left(\frac{n}{b}\right) + d(n) \end{aligned}$$

$$\begin{aligned}
&= a^2[aT(\frac{n}{b^3})+d(\frac{n}{b^2})]+ad(\frac{n}{b})+d(n)=a^3T(\frac{n}{b^3})+a^2d(\frac{n}{b^2})+ad(\frac{n}{b})+d(n) \\
&= \dots \\
&= a^i T(\frac{n}{b^i}) + \sum_{j=0}^{i-1} a^j d(\frac{n}{b^j})
\end{aligned}$$

Ahora, suponiendo que $n = b^k$, se puede utilizar el hecho de que $T(n/b^k) = T(1) = 1$, para tener de lo anterior, con $i = k$, la fórmula

$$T(n) = a^k + \sum_{j=0}^{k-1} a^j d(b^{k-j}) \quad (9.16)$$

Si se aplica el hecho de que $k = \log_b n$, el primer término de (9.16) puede escribirse como $a^{\log_b n}$ o, de forma equivalente, $n^{\log_b a}$ (se toman logaritmos de base b de ambas expresiones para ver que son lo mismo). Esta expresión es n a una potencia constante. Por ejemplo, en el caso de *mergesort*, donde $a = b = 2$, el primer término es n . En general, cuanto mayor sea a , es decir, cuantos más subproblemas haya que resolver, tanto mayor será el exponente; cuanto mayor sea b , es decir, cuanto menor sea cada subproblema, tanto menor será el exponente.

Soluciones homogéneas y particulares

Es interesante ver los diferentes papeles que desempeñan los dos términos de (9.16). El primero, a^k o $n^{\log_b a}$, se conoce como *solución homogénea*, en analogía con la terminología de las ecuaciones diferenciales. La solución homogénea es la solución exacta cuando $d(n)$, conocida como *función motriz*, es 0 para toda n . En otras palabras, la solución homogénea representa el costo de resolver todos los subproblemas, aunque puedan combinarse sin costo.

Por otra parte, el segundo término de (9.16) comprende el costo de creación de los subproblemas y la combinación de sus resultados. Este término se denomina *solución particular*, que está afectada tanto por la función motriz como por el número y tamaño de los subproblemas. Como regla general, si la solución homogénea es mayor que la función motriz, la solución particular tendrá la misma tasa de crecimiento que la solución homogénea. Si la función motriz crece más rápido que la solución homogénea en más que n^ϵ para alguna $\epsilon > 0$, la solución particular tendrá la misma tasa de crecimiento que la función motriz. Si la función motriz tiene la misma tasa de crecimiento que la solución homogénea, o crece más rápido que $\log^k n$ como mucho para alguna k , entonces la solución particular crecerá $\log n$ veces la función motriz.

Es importante reconocer que cuando se buscan mejoras en un algoritmo es necesario saber si la solución homogénea es mayor que la función motriz. Por ejemplo, si la solución homogénea es mayor, prácticamente no tendrá efecto encontrar una forma más rápida de combinar los subproblemas en la eficiencia de todo el al-

goritmo. Lo mejor, en este caso, es encontrar una forma de dividir el problema en menos o menores subproblemas. Eso afectará a la solución homogénea y reducirá el tiempo total de ejecución.

Si la función motriz excede a la solución homogénea, entonces es necesario tratar de reducir la función motriz. Por ejemplo, en el caso de *mergesort*, donde $a = b = 2$, y $d(n) = cn$, la solución particular es $O(n \log n)$. Sin embargo, reducir $d(n)$ a una función ligeramente sublineal, por ejemplo, $n^{0.9}$, hará que la solución particular sea también menos que lineal y que se reduzca el tiempo de ejecución total a $O(n)$, que es la solución homogénea †.

Funciones multiplicativas

La solución particular de (9.16) es difícil de evaluar, aun sabiendo lo que es $d(n)$. Sin embargo, para ciertas funciones $d(n)$ comunes, se puede resolver (9.16) con exactitud, y hay otras para las cuales se puede conseguir una buena cota superior. Se dice que una función f en enteros es *multiplicativa* si $f(xy) = f(x)f(y)$ para todos los enteros positivos x e y .

Ejemplo 9.2. Las funciones multiplicativas de mayor interés son de la forma n^a para cualquier a positiva. Para demostrar que $f(n) = n^a$ es multiplicativa, sólo hay que observar que $(xy)^a = x^a y^a$. □

Ahora, si $d(n)$ de (9.16) es multiplicativa, entonces $d(b^{k-j}) = (d(b))^{k-j}$, y la solución particular de (9.16) es

$$\begin{aligned} \sum_{j=0}^{k-1} a^j (d(b))^{k-j} &= d(b)^k \sum_{j=0}^{k-1} \left(\frac{a}{d(b)} \right)^j \\ &= d(b)^k \frac{\left(\frac{a}{d(b)} \right)^k - 1}{\frac{a}{d(b)} - 1} \\ &= \frac{a^k - d(b)^k}{\frac{a}{d(b)} - 1} \end{aligned} \tag{9.17}$$

Hay tres casos a considerar, dependiendo de si a es mayor, menor o igual que $d(b)$.

† Pero no debe esperarse descubrir una manera de combinar dos listas clasificadas de $n/2$ elementos en un tiempo menor que el lineal; en ese caso, no sería posible siquiera ver todos los elementos de la lista.

1. Si $a > d(b)$, la fórmula (9.17) es $O(a^k)$, que como se recuerda es $n^{\log_b a}$, ya que $k = \log_b n$. En este caso, las soluciones particular y homogénea son iguales, y sólo dependen de a y b , y no de la función motriz d . Así, las mejoras en el tiempo de ejecución deben proceder de disminuir a o aumentar b ; la disminución de $d(n)$ no es muy útil.
2. Si $a < d(b)$, (9.17) es $O(d(b)^k)$ o, en forma equivalente, $O(n^{\log_b d(b)})$. En este caso, la solución particular excede a la homogénea, y se puede dirigir la atención también hacia la función motriz $d(n)$, además de a y b , para obtener mejoras. Obsérvese el importante caso especial en que $d(n) = n^a$. Entonces, $d(b) = b^a$, y $\log_b(b^a) = a$. Así, la solución particular es $O(n^a)$ o bien $O(d(n))$.
3. Si $a = d(b)$, se reconsideran los cálculos implicados en (9.17), pues la fórmula para la suma de una serie geométrica no es ahora apropiada. En este caso,

$$\begin{aligned}
 \sum_{j=0}^{k-1} a^j (d(b))^{k-j} &= d(b)^k \sum_{j=0}^{k-1} \left(\frac{a}{d(b)} \right)^j \\
 &= d(b)^k \sum_{j=0}^{k-1} 1 \\
 &= d(b)^k k \\
 &= n^{\log_b d(b)} \log_b n
 \end{aligned} \tag{9.18}$$

Como $a = d(b)$, la solución particular dada por (9.18) es $\log_b n$ veces la solución homogénea, y de nuevo la solución particular excede a la homogénea. En el caso especial $d(n) = n^a$, (9.18) se reduce a $O(n^a \log n)$, por observaciones similares a las del caso (2).

Ejemplo 9.3. Considérense las siguientes recurrencias, con $T(1) = 1$.

1. $T(n) = 4T(n/2) + n$
2. $T(n) = 4T(n/2) + n^2$
3. $T(n) = 4T(n/2) + n^3$

En cada caso, $a = 4$, $b = 2$, y la solución homogénea es n^2 . En la ecuación (1), con $d(n) = n$, se tiene $d(b) = 2$. Como $a = 4 > d(b)$, la solución particular también es n^2 , y $T(n)$ es $O(n^2)$ en (1).

En la ecuación (3), $d(n) = n^3$, $d(b) = 8$, y $a < d(b)$. Así, la solución particular es $O(n^{\log_2 d(b)}) = O(n^3)$, y $T(n)$ de la ecuación (3) es $O(n^3)$. Se puede deducir que la solución particular es del mismo orden que $d(n) = n^3$, aplicando las observaciones anteriores acerca de los $d(n)$ de la forma n^a y analizando el caso $a < d(b)$ de (9.17). En la ecuación (2), se tiene $d(b) = 4 = a$, con lo que se aplica (9.18). Como $d(n)$ es de la forma n^a , la solución particular y, por tanto, $T(n)$, es $O(n^2 \log n)$. □

Otras funciones motrices

Existen otras funciones motrices no multiplicativas, por medio de las cuales se obtienen soluciones para (9.16) e incluso para (9.17). Se considerarán dos ejemplos. El primero generaliza cualquier función que sea el producto de una función multiplicativa y una constante mayor o igual que uno. La segunda es típica de un caso donde es preciso examinar (9.16) con detalle y obtener una cota superior ajustada a la solución particular.

Ejemplo 9.4. Considérese

$$T(1) = 1$$

$$T(n) = 3T(n/2) + 2n^{1.5}$$

Ahora, $2n^{1.5}$ no es multiplicativa, pero $n^{1.5}$ sí lo es. Sea $U(n) = T(n)/2$ para toda n . Entonces,

$$U(1) = 1/2$$

$$U(n) = 3U(n/2) + n^{1.5}$$

La solución homogénea, si $U(1)$ fuera 1, sería $n^{\log 3} = n^{1.59}$; como $U(1) = 1/2$, se demuestra con facilidad que la solución homogénea es $n^{1.59}/2$; y es $O(n^{1.59})$. Para la solución particular se ignora el hecho de que $U(1) \neq 1$, dado que al incrementar $U(1)$ seguramente no se reducirá la solución particular. Como $a = 3$, $b = 2$ y $b^{1.5} = 2.82 < a$, la solución particular también es $O(n^{1.59})$, que es la tasa de crecimiento de $U(n)$. Como $T(n) = 2U(n)$, $T(n)$ también es $O(n^{1.59})$ o $O(n^{\log 3})$. $\square$

Ejemplo 9.5. Considérese

$$T(1) = 1$$

$$T(n) = 2T(n/2) + n \log n$$

Es fácil observar que la solución homogénea es n , pues $a = b = 2$. Sin embargo, $d(n) = n \log n$ no es multiplicativa, y se debe sumar la fórmula de la solución particular de (9.16) por medios apropiados. Esto es, se desea evaluar

$$\begin{aligned} \sum_{j=0}^{k-1} 2^{2^{k-j}} \log(2^{k-j}) &= 2^k \sum_{j=0}^{k-1} (k-j) \\ &= 2^{k-1} k(k+1) \end{aligned}$$

Como $k = \log n$, la solución particular es $O(n \log^2 n)$, y esta solución, al ser mayor que la homogénea, también es el valor que se obtiene de $T(n)$. $\square$

Ejercicios

- 9.1 Escribanse algunas de las ecuaciones de recurrencia para las complejidades de tiempo y espacio correspondientes al siguiente algoritmo, suponiendo que n es una potencia de 2.

```

function camino (s, t, n: integer): boolean;
begin
  if n = 1 then
    if arista (s, t) then
      return (true)
    else
      return (false);
  { si se llega aquí es que n > 1 }
  for i: = 1 to n do
    if camino (s, i, n div 2) and camino (i, t, n div 2) then
      return (true);
  return (false)
end; { camino }

```

La función $arista(i, j)$ devuelve verdadero si los vértices i y j de un grafo de n vértices están conectados por una arista o si $i = j$; de lo contrario, $arista(i, j)$ devuelve falso. ¿Qué hace el programa?

- 9.2 Resuélvanse las siguientes recurrencias, donde $T(1) = 1$ y $T(n)$ para $n \geq 2$ satisface:
- $T(n) = 3T(n/2) + n$
 - $T(n) = 3T(n/2) + n^2$
 - $T(n) = 8T(n/2) + n^3$
- 9.3 Resuélvanse las siguientes recurrencias, donde $T(1) = 1$ y $T(n)$ para $n \geq 2$ satisface:
- $T(n) = 4T(n/3) + n$
 - $T(n) = 4T(n/3) + n^2$
 - $T(n) = 9T(n/3) + n^2$
- 9.4 Obténgase las cotas Θ y Ω en los $T(n)$ definidos por las siguientes recurrencias. Supóngase que $T(1) = 1$
- $T(n) = T(n/2) + 1$
 - $T(n) = 2T(n/2) + \log n$
 - $T(n) = 2T(n/2) + n$
 - $T(n) = 2T(n/2) + n^2$

***9.5** Resuélvanse las siguientes recurrencias suponiendo una solución y comprobando la respuesta

a) $T(1) = 2$

$$T(n) = 2T(n-1) + 1 \text{ para } n \geq 2$$

b) $T(1) = 1$

$$T(n) = 2T(n-1) + n \text{ para } n \geq 2$$

9.6 Verifíquense las respuestas del ejercicio 9.5 resolviendo las recurrencias por sustitución repetida.

9.7 Generalícese el ejercicio 9.6 resolviendo todas las recurrencias de la forma

$$T(1) = 1$$

$$T(n) = aT(n-1) + d(n) \text{ para } n \geq 1$$

en función de a y $d(n)$.

***9.8** Supóngase en el ejercicio 9.7 que $d(n) = c^n$ para alguna constante $c \geq 1$. ¿Qué dependencia tiene la solución de $T(n)$ de la relación entre a y c ? ¿Qué es $T(n)$?

****9.9** Resuélvase para $T(n)$:

$$T(1) = 1$$

$$T(n) = \sqrt{n}T(\sqrt{n}) + n \text{ para } n \geq 2$$

9.10 Encuéntrense expresiones en su forma cerrada para las siguientes sumas.

a) $\sum_{i=0}^n i$

b) $\sum_{i=0}^n i^k$

c) $\sum_{i=0}^n 2^i$

d) $\sum_{i=0}^n \binom{n}{i}$

***9.11** Muéstrase que el número de órdenes distintos en que se puede multiplicar una secuencia de n matrices, está dado por la recurrencia

$$T(1) = 1$$

$$T(n) = \sum_{i=1}^{n-1} T(i)T(n-i)$$

Muéstrase que $T(n+1) = \frac{1}{n+1} \binom{2n}{n}$. Los $T(n)$ se conocen como *números de Catalán*.

****9.12** Muéstrase que el número de comparaciones requeridas para clasificar n elementos con clasificación por intercalación (*mergesort*) está dado por

$$T(1) = 0$$

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + n - 1$$

donde $\lfloor x \rfloor$ denota la parte entera de x , y $\lceil x \rceil$, el entero más pequeño $\geq x$. Muéstrese que la solución a esta recurrencia es

$$T(n) = n \lceil \log n \rceil - 2^{\lceil \log n \rceil} + 1$$

- 9.13 Muéstrese que el número de funciones booleanas de n variables está dado por la recurrencia

$$T(1) = 4$$

$$T(n) = (T(n-1))^2$$

Resuélvase para $T(n)$.

- **9.14 Muéstrese que el número de árboles binarios de altura $\leq n$ está dado por la recurrencia

$$T(1) = 1$$

$$T(n) = (T(n-1))^2 + 1$$

Muéstrese que $T(n) = \lfloor k^{2^n} \rfloor$ para alguna constante k . ¿Cuál es el valor de k ?

Notas bibliográficas

Bentley, Haken, y Saxe [1978], Greene y Knuth [1983], Liu [1968] y Lueker [1980] contienen material adicional sobre la solución de recurrencias. En Aho y Sloane [1973] se demuestra que muchas recurrencias no lineales de la forma $T(n) = (T(n-1))^2 + g(n)$ tienen una solución de la forma $T(n) = \lfloor k^{2^n} \rfloor$ donde k es una constante, como en el ejercicio 9.14.

10

Técnicas de diseño
de algoritmos

A través de los años, los científicos de la computación han identificado diversas técnicas generales que a menudo producen algoritmos eficientes para la resolución de muchas clases de problemas. Este capítulo presenta algunas de las técnicas más importantes como dividir para vencer, programación dinámica, técnicas ávidas, el método de retroceso y búsqueda local. En el intento de diseñar un algoritmo para resolver un problema dado, a menudo es útil plantear cuestiones como «¿qué clase de solución se puede obtener de la programación dinámica, del enfoque ávido, de la técnica dividir para vencer o de otra técnica estándar?».

Sin embargo, debe subrayarse que hay algunos problemas, como los NP completos, para los cuales ni éstas ni otras técnicas conocidas producirán soluciones eficientes. Cuando se encuentra algún problema de este tipo, suele ser útil determinar si las entradas al problema tienen características especiales que se puedan explotar en la búsqueda de una solución, o si puede usarse alguna solución aproximada sencilla, en vez de la solución exacta, difícil de calcular.

10.1 Algoritmos dividir para vencer

Tal vez la técnica más importante y aplicada para el diseño de algoritmos eficientes sea la estrategia llamada dividir para vencer, que consiste en la descomposición de un problema de tamaño n en problemas más pequeños, de modo que a partir de la solución de dichos problemas sea posible construir con facilidad una solución al problema completo. Ya se han visto varias aplicaciones de esta técnica, como la clasificación por intercalación o los árboles binarios de búsqueda.

Para ilustrar el método, considérese el conocido acertijo de las «torres de Hanoi». Consta de tres postes A , B y C . Inicialmente, el poste A tiene cierta cantidad de discos de distintos tamaños, con el más grande en la parte inferior y otros sucesivamente más pequeños encima, como se muestra en la figura 10.1. El objeto del acertijo es pasar un disco a la vez de un poste a otro, sin colocar nunca un disco grande sobre otro más pequeño, hasta que todos los discos estén en el poste B .

Pronto se descubre que el acertijo puede resolverse con el sencillo algoritmo siguiente. Se imaginan los postes dispuestos en un triángulo. Con un número de movimientos impar, se mueve el disco más pequeño hacia un poste en el sentido de las manecillas del reloj. Con un número de movimientos pares, se hace el único movimiento válido que no implique al disco más pequeño.

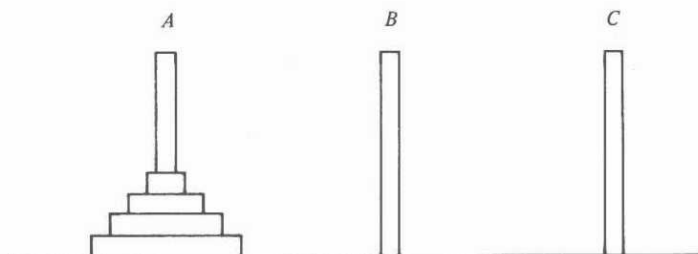


Fig. 10.1. Posición inicial en el acertijo de las torres de Hanoi.

El algoritmo anterior es conciso y correcto, pero es difícil entender por qué funciona y de descubrir intuitivamente. En cambio, considérese el siguiente enfoque de dividir para vencer. El problema de pasar los n discos más pequeños de A a B puede considerarse compuesto de dos problemas de tamaño $n - 1$. Primero se mueven los $n - 1$ discos más pequeños del poste A al C , dejando el n -ésimo disco más pequeño en el poste A . Se mueve ese disco de A a B . Después se pasan los $n - 1$ discos más pequeños de C a B . El movimiento de los $n - 1$ discos más pequeños se efectuará por medio de la aplicación recursiva del método. Como los n discos implicados en los movimientos son más pequeños que los demás discos, no es necesario preocuparse de los que se encuentran debajo de ellos en los postes A , B o C . Aunque el movimiento real de los discos individuales no es obvio y la simulación a mano es difícil debido al apilamiento de llamadas recursivas, el algoritmo es conceptualmente fácil de entender, de probar que es correcto y, tal vez, de considerarlo como primera opción. Es probable que la facilidad de descubrir los algoritmos de división haga que esta técnica sea tan importante, aunque en muchos casos los algoritmos son también más eficientes que otros más convencionales †.

Problema de la multiplicación de enteros grandes

Considérese el problema de la multiplicación de dos enteros X e Y de n bits. Recuérdese que el algoritmo para la multiplicación de enteros de n bits (o n dígitos), que se suele enseñar en la escuela primaria, implica el cálculo de n productos parciales de tamaño n , de aquí que sea un algoritmo $O(n^2)$, si se toman las multiplicaciones y las adiciones de un sólo bit o dígito como un paso. Un enfoque de clasificación del resultado por división en la multiplicación de enteros dividiría X e Y en dos enteros de $n/2$ bits cada uno, como se muestra en la figura 10.2. (Por simplicidad, se supone que n es una potencia de 2.)

† En el caso de las torres de Hanoi, el algoritmo dividir para vencer es en realidad el mismo que el dado inicialmente.

$$\begin{array}{ll}
 X := \begin{array}{|c|c|} \hline A & B \\ \hline \end{array} & X = A2^{n/2} + B \\
 Y := \begin{array}{|c|c|} \hline C & D \\ \hline \end{array} & Y = C2^{n/2} + D
 \end{array}$$

Fig. 10.2. División de un entero de n bits en segmentos de $n/2$ bits.

El producto de X e Y puede escribirse como

$$XY = AC2^n + (AD+BC) 2^{n/2} + BD \quad (10.1)$$

Si se evalúa XY de esta forma directa, es necesario realizar cuatro multiplicaciones de enteros de $n/2$ bits (AC , AD , BC , BD), tres sumas de enteros con un máximo de $2n$ bits (correspondientes a los tres signos $+$ de (10.1)), y dos desplazamientos (multiplicaciones por 2^n y por $2^{n/2}$). Como esas sumas y desplazamientos requieren $O(n)$ pasos, se puede escribir la siguiente recurrencia para $T(n)$, el número total de operaciones de bits necesarias para multiplicar enteros de n bits de acuerdo con (10.1).

$$T(1) = 1$$

$$T(n) = 4T(n/2) + cn \quad (10.2)$$

Por un razonamiento similar al del ejemplo 9.4, se toma la constante c de (10.2) como 1, de modo que la función motriz $d(n)$ es tan sólo n , y se deduce que la solución homogénea y la particular son $O(n^2)$.

En caso de que la fórmula (10.1) se utilice para multiplicar enteros, la eficiencia asintótica no es mayor que para el método de la escuela primaria. Pero recuérdese que para ecuaciones como (10.2) se consigue una mejora asintótica si se reduce el número de subproblemas. Puede ser sorprendente que se haga eso, pero considérese la siguiente fórmula para multiplicar X por Y .

$$XY = AC2^n + [(A-B)(D-C) + AC + BD]2^{n/2} + BD \quad (10.3)$$

Aunque (10.3) parece más complicada que (10.1), sólo requiere tres multiplicaciones de enteros con $(n/2)$ bits, AC , BD y $(A-B)(D-C)$, seis sumas o restas y dos desplazamientos. Como todas las operaciones, excepto las multiplicaciones, requieren $O(n)$ pasos, el tiempo $T(n)$ para multiplicar enteros de n bits por (10.3) está dado por

$$T(1) = 1$$

$$T(n) = 3T(n/2) + cn$$

cuya solución es $T(n) = O(n^{\log_2 3}) = O(n^{1.59})$.

El algoritmo completo, incluyendo los detalles implicados por el hecho de que (10.3) requiere multiplicaciones de enteros de $(n/2)$ bits tanto negativos como positivos, está dado en la figura 10.3. Obsérvese que las líneas (8) a (11) se realizan co-

piando bits, y la multiplicación por 2^n y $2^{n/2}$ de la línea (16), por desplazamientos. Del mismo modo, la multiplicación por s de la línea (16) tan sólo introduce el signo adecuado en el resultado.

```

function mult ( X, Y, n: integer ): integer;
  { X e Y son enteros con signo  $\leq 2^n$ .
    n es una potencia de 2. La función devuelve XY }
  var
    s: integer; { contiene el signo de XY }
    m1, m2, m3: integer; { contiene los tres productos }
    A, B, C, D: integer; { contiene las mitades izquierda y derecha de X e Y }
  begin
    (1)  s := sign(X) * sign(Y);
    (2)  X := abs(X);
    (3)  Y := abs(Y); { hace positivos a X e Y }
    (4)  if n = 1 then
    (5)    if (X = 1) and (Y = 1) then
    (6)      return (s)
    (7)    else
    (8)      return (0)
    (9)    else begin
    (10)     A := n/2 bits izquierdos de X;
    (11)     B := n/2 bits derechos de X;
    (12)     C := n/2 bits izquierdos de Y;
    (13)     D := n/2 bits derechos de Y;
    (14)     m1 := mult(A, C, n/2);
    (15)     m2 := mult(A - B, D - C, n/2);
    (16)     m3 := mult(B, D, n/2);
    (17)     return (s * (m1*2n + (m1 + m2 + m3) * 2n/2 + m3))
    (18)   end
  end; { mult }
  
```

Fig. 10.3. Algoritmo dividir para vencer de la multiplicación de enteros.

Obsérvese que el algoritmo de la figura 10.3 es asintóticamente más rápido que el método estudiado en la escuela primaria, al requerir $O(n^{1.59})$ pasos en vez de $O(n^2)$. Puede plantearse la pregunta de por qué no se imparte en la escuela primaria si es mejor. Existen dos respuestas: primero, aunque es fácil de implantar en un computador, la descripción del algoritmo es lo bastante compleja que si se intenta enseñarlo en la escuela primaria, los estudiantes no aprenderán a multiplicar. Más aún, se han ignorado las constantes de proporcionalidad. Mientras que el procedimiento *mult* de la figura 10.3 es asintóticamente superior al método usual, las constantes son tales que para problemas pequeños (de hecho hasta de 500 bits) el método de la escuela primaria es mejor, y en raras ocasiones se pide a estudiantes de ese nivel que multipliquen números de esa magnitud.

Programación de torneos de tenis

La técnica de dividir para vencer tiene varias aplicaciones, no sólo en el diseño de algoritmos, sino también en el diseño de circuitos, construcción de demostraciones matemáticas y en otros campos del quehacer humano. Se brinda un ejemplo para ilustrar esto. Considérese el diseño del programa de un torneo de tenis para $n = 2^k$ jugadores. Cada jugador debe enfrentarse a todos los demás, y debe tener un encuentro diario durante $n - 1$ días, que es el número mínimo de días necesarios para completar el torneo.

El programa del torneo es una tabla de n filas por $n - 1$ columnas cuya posición en fila i y columna j es el jugador con quien debe contender i el j -ésimo día.

El enfoque dividir para vencer construye un programa para la mitad de los jugadores, que está diseñado por una aplicación recursiva del algoritmo buscando un programa para una mitad de esos jugadores, y así sucesivamente. Cuando se llega hasta dos jugadores, se tiene el caso base y simplemente se emparejan.

Supóngase que hay ocho jugadores. El programa para los jugadores 1 a 4 ocupa la esquina superior izquierda (4 filas por 3 columnas) del programa que se está construyendo. La esquina inferior izquierda (4 filas por 3 columnas) debe enfrentar a los jugadores con numeración más alta (5 a 8). Este subprograma se obtiene agregando 4 a cada entrada de la esquina superior izquierda.

Ahora se ha simplificado el problema, y lo único que falta es enfrentar a los jugadores de baja numeración con los de alta numeración. Esto es fácil si los jugadores 1 a 4 contienden con los 5 a 8, respectivamente, en el día 4, y se permutan cíclicamente 5 a 8 en los días siguientes. El proceso se ilustra en la figura 10.4. El lector debe ser capaz de generalizar las ideas de esta figura para obtener un programa de 2^k jugadores para cualquier k .

Balanceo de subproblemas

En el diseño de algoritmos siempre hay que afrontar varios compromisos. Ha surgido la regla de que suele ser ventajoso balancear los costos de competencia siempre que sea posible. Por ejemplo, en el capítulo 5 se vio que los árboles 2-3 balanceaban los costos de búsqueda con los de inserción, mientras que los métodos más directos requieren $O(n)$ pasos por cada búsqueda o por cada inserción, aun cuando la otra operación pueda efectuarse en un número constante de pasos.

En forma semejante, para los algoritmos de dividir para vencer, casi siempre es mejor que los subproblemas tengan un tamaño aproximadamente igual. Por ejemplo, la clasificación por inserciones puede considerarse como la división de un problema en dos subproblemas, uno de tamaño 1 y otro de tamaño $n - 1$, con un costo máximo de n pasos para combinarlos. Esto da la recurrencia

$$T(n) = T(1) + T(n - 1) + n$$

que tiene una solución $O(n^2)$. La clasificación por intercalación, por otra parte, divide el problema en dos subproblemas de tamaño $n/2$ y tiene un rendimiento

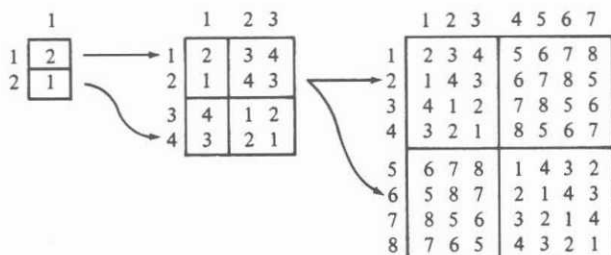


Fig. 10.4. Torneo para ocho jugadores.

$O(n \log n)$. Como principio general, a menudo resulta que dividir un problema en subproblemas iguales o casi iguales es un factor crucial para la obtención de un buen rendimiento.

10.2 Programación dinámica

A menudo sucede que no hay manera de dividir un problema en un pequeño número de subproblemas cuya solución pueda combinarse para resolver el problema original. En tales casos, se puede intentar dividir el problema en tantos subproblemas como sea necesario, dividir cada subproblema en subproblemas más pequeños, y así sucesivamente. Si eso es todo lo que se hace, quizá se produzca un algoritmo de tiempo exponencial.

No obstante, con frecuencia, sólo hay un número polinomial de subproblemas, de aquí que se deba resolver algún subproblema muchas veces. Si, en cambio, se conserva la solución a cada subproblema resuelto, y tan sólo se toma la respuesta cuando se requiere, se obtiene un algoritmo de tiempo polinomial.

Desde el punto de vista de la realización, algunas veces es más fácil crear una tabla de las soluciones de todos los subproblemas que se tengan que resolver. Se rellena la tabla sin tener en cuenta si se necesita realmente un subproblema particular en la solución total. La formación de la tabla de subproblemas para alcanzar una solución a un problema dado se denomina *programación dinámica*, nombre procedente de la teoría de control.

La forma de un algoritmo de programación dinámica puede variar, pero hay un esquema común: una tabla a rellenar y un orden en el cual se hacen las entradas. Se ilustrarán las técnicas mediante dos ejemplos, el cálculo de apuestas en un encuentro como la Serie Mundial de Béisbol y el «problema de la triangulación».

Apuestas en la Serie Mundial de Béisbol

Supóngase que dos equipos, *A* y *B*, tienen un enfrentamiento para ver quién es el primero en ganar n partidos para una n en particular. La Serie Mundial es uno de

tales enfrentamientos, con $n = 4$. Se supone que A y B son igualmente competentes, de modo que cada uno tiene un 50% de oportunidades de ganar cualquier partido. Sea $P(i, j)$ la probabilidad de que A gane el encuentro, considerando que A necesita i partidos para ganar, y B necesita j . Por ejemplo, si los Dodgers han ganado dos partidos y los Yankees uno, entonces $i = 2, j = 3$, y se observa que $P(2, 3)$ es $11/16$.

Para calcular $P(i, j)$, se emplea una ecuación de recurrencia en dos variables. Primero, si $i = 0$ y $j > 0$, el equipo A ya ha ganado el encuentro, por lo que $P(0, j) = 1$. Igualmente, $P(i, 0) = 0$ para $i > 0$. Si i y j son mayores que cero, deberá jugarse al menos otro partido más y los dos equipos ganan la mitad de las veces. Así, $P(i, j)$ debe ser el promedio de $P(i-1, j)$ y $P(i, j-1)$, siendo la primera de ellas la probabilidad de que A gane el encuentro si gana el siguiente partido, y la segunda, la probabilidad de que A gane el encuentro aun cuando pierda el siguiente partido. En resumen:

$$\begin{aligned} P(i, j) &= 1 \quad \text{si } i = 0 \text{ y } j > 0 \\ &= 0 \quad \text{si } i > 0 \text{ y } j = 0 \\ &= (P(i-1, j) + P(i, j-1))/2 \quad \text{si } i > 0 \text{ y } j > 0 \end{aligned} \quad (10.4)$$

Si se utiliza (10.4) recursivamente como una función, se demuestra que $P(i, j)$ requiere un tiempo no mayor que $O(2^{i+j})$. Sea $T(n)$ el tiempo máximo requerido por una llamada a $P(i, j)$, donde $i + j = n$. Entonces, de (10.4),

$$T(1) = c$$

$$T(n) = 2T(n-1) + d$$

para algunas constantes c y d . Es útil verificar por los medios analizados en el capítulo anterior, que $T(n) \leq 2^{n-1}c + (2^{n-1} - 1)d$, lo cual es $O(2^n)$ u $O(2^{i+j})$.

Así, se ha obtenido una cota superior exponencial para el tiempo requerido por el cálculo recursivo de $P(i, j)$. Sin embargo, para tener la convicción de que la fórmula recursiva para $P(i, j)$ es una mala forma de calcularla, se toma una cota inferior omega mayúscula. Se deja como ejercicio demostrar que al llamar a $P(i, j)$, el número total de llamadas a P que se hace es $\binom{i+j}{i}$, el número de formas de escoger i objetos a partir de $i + j$. Si $i = j$, ese número es $\Omega(2^n/\sqrt{n})$, donde $n = i + j$. Así, $T(n)$ es $\Omega(2^n/\sqrt{n})$ y, de hecho, se demuestra que también es $O(2^n/\sqrt{n})$. Mientras $2^n/\sqrt{n}$ crece asintóticamente más lento que 2^n , la diferencia no es muy grande, y $T(n)$ crece demasiado rápido como para que el cálculo recursivo de $P(i, j)$ sea práctico.

El problema con el cálculo recursivo es que se calcula la misma $P(i, j)$ muchas veces. Por ejemplo, si se desea obtener $P(2, 3)$, se calcula, por (10.4), $P(1, 3)$ y $P(2, 2)$. $P(1, 3)$ y $P(2, 2)$ requieren el cálculo de $P(1, 2)$, por lo que se calcula ese valor dos veces.

Una forma mejor de calcular $P(i, j)$ es llenar la tabla sugerida en la figura 10.5. La fila inferior está llena de ceros y la columna más a la derecha está llena de unos, debido a las dos primeras filas de (10.4). Por la última fila de (10.4), cada una de las otras entradas es el promedio de la entrada de abajo y de la entrada de la derecha. Así, una forma apropiada de llenar la tabla es proceder en diagonales a partir

la esquina inferior derecha, y procediendo hacia arriba y hacia la izquierda en diagonales que representan las posiciones con valor constante de $i + j$, como se sugiere en la figura 10.6. Este programa se presenta en la figura 10.7, suponiendo que opera en un arreglo bidimensional P de tamaño apropiado.

	1/2	21/32	13/16	15/16	1	4
	11/32	1/2	11/16	7/8	1	3
	3/16	5/16	1/2	3/4	1	2
	1/16	1/8	1/4	1/2	1	1
	0	0	0	0		0
	4	3	2	1	0	
	$\leftarrow i$					

Fig. 10.5. Tabla de apuestas.

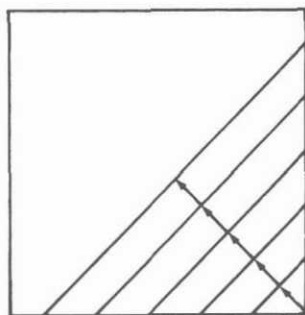


Fig. 10.6. Patrón de computación.

El análisis de la función *apuestas* es fácil. El ciclo de las líneas (4) a (5) requiere un tiempo $O(s)$, y domina el tiempo $O(1)$ de las líneas (2) a (3). Así, el ciclo externo requiere un tiempo $O(\sum_{j=1}^n s)$ u $O(n^2)$, donde $i + j = n$. Con ello, la programación dinámica requiere un tiempo $O(n^2)$, comparado con $O(2^n/\sqrt{n})$ para el enfoque directo. Puesto que $2^n/\sqrt{n}$ crece mucho más rápido que n^2 , casi siempre es preferible utilizar la programación dinámica que el enfoque recursivo.

```
function apuestas ( i, j: integer ) : real;
var
    s, k: integer;
begin
```



```

(1)      for s := 1 to i + j do begin
           { calcular la diagonal de las posiciones cuyos índices se suman a s }
(2)      P[0, s] := 1.0;
(3)      P[s, 0] := 0.0;
(4)      for k := 1 to s - 1 do
(5)      P[k, s - k] := (P[k - 1, s - k] + P[k, s - k - 1])/2.0
           end;
(6)      return (P[i, j])
end; { apuestas }

```

Fig. 10.7. Cálculo de apuestas.

Problema de la triangulación

Como otro ejemplo de programación dinámica, considérese el problema de la *triangulación* de un polígono. Se dan los vértices de un polígono y una medida de la distancia entre cada par de vértices. La distancia puede ser la normal (euclidiana) en el plano, o puede ser una función de costo arbitraria dada por una tabla. El problema consiste en seleccionar un conjunto de *cuerdas* (líneas entre vértices no adyacentes) de modo que dos cuerdas no se crucen entre sí y que todo el polígono quede dividido en triángulos. La longitud total (distancia entre puntos finales) de las cuerdas seleccionadas debe ser mínima. Ese conjunto de cuerdas se llama *triangulación minimal*.

Ejemplo 10.1. La figura 10.8 muestra un polígono de siete lados y las coordenadas (x, y) de sus vértices. La función de distancia es la distancia normal euclidiana. Una triangulación no minimal, se muestra por medio de las líneas de puntos. Su costo es la suma de las longitudes de las cuerdas (v_0, v_2) , (v_0, v_3) , (v_0, v_5) y (v_3, v_5) , o $\sqrt{8^2+16^2} + \sqrt{15^2+16^2} + \sqrt{22^2+2^2} + \sqrt{7^2+14^2} = 77.56$. $\square$

Además de ser interesante por sí mismo, el problema de la triangulación tiene diversas aplicaciones útiles. Por ejemplo, Fuchs, Kedem, y Useton [1977] usaron una generalización del problema de la triangulación para el propósito siguiente. Considérese el problema de sombrear la imagen bidimensional de un objeto cuya superficie está definida por una colección de puntos en un espacio tridimensional. La fuente de luz proviene de una dirección dada, y el brillo de un punto en la superficie depende de los ángulos entre la dirección de la luz, la dirección de la vista del observador y una perpendicular a la superficie en ese punto. Para estimar la dirección de la superficie en un punto, se calcula una triangulación mínima de los puntos que definen la superficie.

Cada triángulo define un plano en el espacio tridimensional, y dado que se encontró una triangulación mínima, es de esperarse que los triángulos sean muy pequeños. Es fácil encontrar la dirección de una perpendicular a un plano, así que se calcula la intensidad de la luz para los puntos de cada triángulo en el supuesto de que la superficie puede considerarse como un plano triangular en una región dada. Si los triángulos no son suficientemente pequeños como para hacer que la intensidad de la luz aparezca suave, entonces obtener el promedio local puede mejorar la imagen.

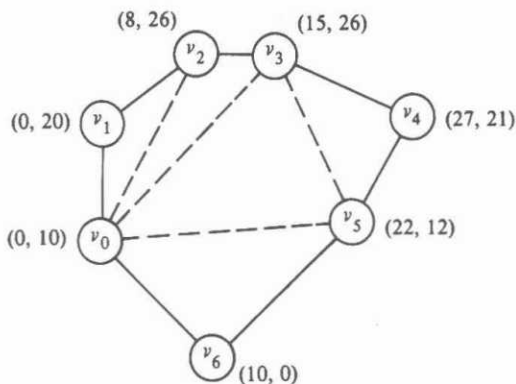


Fig. 10.8. Heptágono y triangulación.

Antes de proseguir con la solución de la programación dinámica al problema de la triangulación, se dejarán establecidas dos observaciones acerca de las triangulaciones que ayudarán a diseñar el algoritmo. Se supone que hay un polígono con n vértices $v_0, v_1, \dots, v_{n-1}$, en el sentido de las manecillas del reloj.

Hecho 1. En cualquier triangulación de un polígono con más de tres vértices, cada par de vértices adyacentes es tocado al menos por una cuerda. Para ver esto, supóngase que ni v_i ni v_{i+1} † fueron tocados por una cuerda. Entonces la región que la arista (v_i, v_{i+1}) limita debe incluir las aristas (v_{i-1}, v_i) , (v_{i+1}, v_{i+2}) y al menos una arista adicional. Esta región no sería un triángulo.

Hecho 2. Si (v_p, v_j) es una cuerda de una triangulación, debe haber algún v_k tal que (v_p, v_k) y (v_k, v_j) sean aristas del polígono o cuerdas. De otra forma, (v_p, v_j) puede limitar una región que no sea un triángulo.

Para iniciar la búsqueda de una triangulación minimal, se escogen dos vértices adyacentes, como v_0 y v_1 . Por los dos hechos, se sabe que en cualquier triangulación y, por tanto, en la triangulación mínima, debe haber un vértice v_k tal que (v_1, v_k) y (v_k, v_0) sean cuerdas o aristas en la triangulación. Así pues, se debe considerar la bondad de la triangulación que se obtendrá después de seleccionar cada valor posible de k . Si el polígono tiene n vértices, se puede hacer un total de $(n - 2)$ selecciones.

Cada selección de k da lugar como máximo a dos subproblemas, que son polígonos formados por una cuerda y las aristas del polígono original que van de un extremo a otro de la cuerda. Por ejemplo, la figura 10.9 muestra los dos subproblemas que resultan al seleccionar el vértice v_3 .

A continuación, se deben encontrar triangulaciones mínimas para los polígonos de la figura 10.9(a) y (b). Lo primero que hay que hacer es considerar otra vez todas

† En lo sucesivo, se considerará que todos los subíndices están calculados módulo n . Así, en la figura 10.8, v_i y v_{i+1} pueden ser v_6 y v_0 , respectivamente, puesto que $n = 7$.

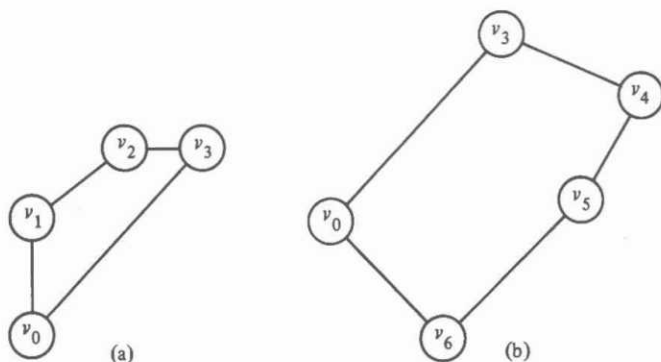


Fig. 10.9. Los dos subproblemas después de seleccionar v_3 .

las cuerdas que parten de dos vértices adyacentes. Por ejemplo, en la solución de la figura 10.9(b), se considera la selección de la cuerda (v_3, v_5) , la cual deja el subproblema (v_0, v_3, v_5, v_6) , un polígono en el cual dos lados, (v_0, v_3) y (v_3, v_5) , son cuerdas del polígono original. Este camino conduce a un algoritmo de tiempo exponencial.

Sin embargo, considerando el triángulo que incluye la cuerda (v_0, v_k) nunca se tienen que considerar polígonos en los cuales más de un lado sean cuerdas del polígono original. El hecho 2 dice que, en la triangulación mínima, la cuerda del subproblema, como (v_0, v_3) de la figura 10.9(b), debe formar un triángulo con uno de los otros vértices. Por ejemplo, si se selecciona v_4 , se forma el triángulo (v_0, v_3, v_4) y el subproblema (v_0, v_4, v_5, v_6) , que sólo tiene una cuerda del polígono original. Al probar con v_5 , se forman los subproblemas (v_3, v_4, v_5) y (v_0, v_5, v_6) , con las cuerdas (v_3, v_5) y (v_0, v_5) nada más.

En general, se define el subproblema de tamaño s partiendo del vértice v_i , denotado S_{is} , como el problema de triangulación minimal para el polígono formado por los s vértices que comienzan en v_i y siguen en el sentido de las manecillas del reloj, esto es, $v_i, v_{i+1}, \dots, v_{i+s-1}$. La cuerda en S_{is} es (v_i, v_{i+s-1}) . Por ejemplo, en la figura 10.9(a) es S_{04} y en la figura 10.9 (b) es S_{35} . Para resolver S_{is} debemos considerar las tres opciones siguientes.

1. Tomar el vértice v_{i+s-2} para formar un triángulo con las cuerdas (v_i, v_{i+s-1}) , (v_i, v_{i+s-2}) y con el tercer lado (v_{i+s-2}, v_{i+s-1}) , y después resolver el subproblema $S_{i, s-1}$.
2. Tomar el vértice v_{i+1} para formar un triángulo con las cuerdas (v_i, v_{i+s-1}) , (v_{i+1}, v_{i+s-1}) y con el tercer lado (v_i, v_{i+1}) , y después resolver el subproblema $S_{i+1, s-1}$.
3. Para alguna k entre 2 y $s-3$, tomar el vértice v_{i+k} y formar un triángulo con lados (v_i, v_{i+k}) , (v_{i+k}, v_{i+s-1}) , y (v_i, v_{i+s-1}) , y después resolver los subproblemas $S_{i, k+1}$ y $S_{i+k, s-k}$.

Si se recuerda que la «solución» a cualquier subproblema de tamaño tres o menor no requiere acción alguna, se puede resumir (1) a (3) diciendo que se toma al-

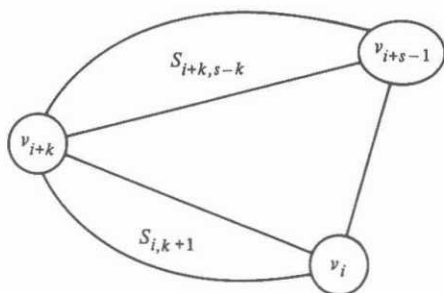


Fig. 10.10. División de $S_{i,s}$ en subproblemas.

gún vértice k entre 1 y $s-2$ y se resuelven los subproblemas $S_{i,k+1}$ y $S_{i+k,s-k}$. La figura 10.10 ilustra esta división en subproblemas.

Si se emplea el algoritmo recursivo implícito en las reglas anteriores para resolver subproblemas de tamaño 4 o mayores se podrá mostrar que cada llamada en un subproblema de tamaño s provoca un total de 3^{s-4} llamadas recursivas, si se «resuelven» directamente los subproblemas de tamaño 3 o menores y sólo se cuentan las llamadas en subproblemas de tamaño 4 o mayores. Así, el número de subproblemas a resolver es exponencial en s . Dado que el problema inicial es de tamaño n , donde n es el número de vértices del polígono dado, el número total de pasos realizados por este procedimiento recursivo es exponencial en n .

Es evidente que todavía hay algo erróneo en este análisis, porque se sabe que sin contar el problema original, sólo hay $n(n-4)$ subproblemas diferentes que necesitan solución, y se representan por $S_{i,s}$, donde $0 \leq i < n$ y $4 \leq s < n$. Es obvio que no todos los subproblemas resueltos por el procedimiento recursivo son diferentes. Por ejemplo, si en la figura 10.8 se escoge la cuerda (v_0, v_3) , y después, en el subproblema de la figura 10.9(b), v_4 , hay que resolver el subproblema S_{44} . Pero también se tendría que resolver este problema si se tomara primero la cuerda (v_0, v_4) , o la cuerda (v_1, v_4) , y entonces, al resolver el subproblema S_{45} , se tomaría el vértice v_0 para completar un triángulo con v_1 y v_4 .

Esto sugiere una forma eficiente de resolver el problema de la triangulación. Se prepara una tabla que contenga el costo $C_{i,s}$ de la triangulación $S_{i,s}$ para toda i y s . Dado que la solución a cualquier problema dado sólo depende de la solución a problemas de tamaño menor, el orden lógico en el cual se debe llenar la tabla es de acuerdo con el tamaño. Esto es, para tamaños $s = 4, 5, \dots, n-1$ se introduce el costo mínimo para los subproblemas $S_{i,s}$, para todos los vértices i . Es conveniente incluir también los problemas de tamaño $0 \leq s < 4$, pero recuérdese que $S_{i,s}$ tiene costo 0 si $s < 4$.

De las reglas anteriores (1) a (3) para encontrar subproblemas, la fórmula para calcular $C_{i,s}$ para $s \geq 4$ es:

$$C_{i,s} = \min_{1 \leq k \leq s-2} \left[C_{i,k+1} + C_{i+k,s-k} + D(v_i, v_{i+k}) + D(v_{i+k}, v_{i+s-1}) \right] \quad (10.5)$$

donde $D(v_p, v_q)$ es la longitud de la cuerda entre los vértices v_p y v_q , si v_p y v_q no son puntos adyacentes en el polígono; $D(v_p, v_q)$ es 0 si v_p y v_q son adyacentes.

Ejemplo 10.2. La figura 10.11 contiene la tabla de costos de S_{is} para $0 \leq i \leq 6$ y $4 \leq s \leq 6$, basada en el polígono y las distancias de la figura 10.8. Los costos de las filas con $s < 3$ son todos cero. Se ha llenado la posición C_{07} , en la columna 0 y la fila para $s = 7$. Este valor, como todos los de la misma fila, representa la triangulación del polígono completo. Para ver esto, obsérvese que se puede, si se desea, considerar la arista (v_0, v_6) como una cuerda de un polígono mayor y el polígono de la figura 10.8 como un subproblema de este polígono, el cual tiene una serie de vértices adicionales que se extienden en el sentido de las manecillas del reloj desde v_6 hasta v_0 . Obsérvese que toda la fila para $s = 7$ tiene el mismo valor que C_{07} , para conservar la exactitud de cálculo.

Como ejemplo, se proporciona la demostración de cómo se introdujo el valor 38.09 de la columna para $i = 6$ y la fila para $s = 5$. De acuerdo con (10.5), el valor de esta posición, C_{65} , es el mínimo de tres sumas correspondientes a $k = 1, 2$ ó 3. Esas sumas son:

$$C_{62} + C_{04} + D(v_6, v_0) + D(v_0, v_3)$$

$$C_{63} + C_{13} + D(v_6, v_1) + D(v_1, v_3)$$

$$C_{64} + C_{22} + D(v_6, v_2) + D(v_2, v_3)$$

7	$C_{07} =$ 75.43						
6	$C_{06} =$ 53.54	$C_{16} =$ 55.22	$C_{26} =$ 57.58	$C_{36} =$ 64.69	$C_{46} =$ 59.78	$C_{56} =$ 59.78	$C_{66} =$ 63.62
5	$C_{05} =$ 37.54	$C_{15} =$ 31.81	$C_{25} =$ 35.49	$C_{35} =$ 37.74	$C_{45} =$ 45.50	$C_{55} =$ 39.98	$C_{65} =$ 38.09
4	$C_{04} =$ 16.16	$C_{14} =$ 16.16	$C_{24} =$ 15.65	$C_{34} =$ 15.65	$C_{44} =$ 22.69	$C_{54} =$ 22.69	$C_{64} =$ 17.89
s	$i=0$	1	2	3	4	5	6

Fig. 10.11. Tabla de C_{is} .

Las distancias requeridas se calculan a partir de las coordenadas de los vértices:

$$D(v_2, v_3) = D(v_6, v_0) = 0$$

(puesto que son aristas del polígono, no cuerdas, y se presentan en forma «gratuita»)

$$D(v_6, v_2) = 26.08$$

$$D(v_1, v_3) = 16.16$$

$$D(v_6, v_1) = 22.36$$

$$D(v_0, v_3) = 21.93$$

Las tres sumas anteriores dan 38.09, 38.52 y 43.97, respectivamente. Se concluye que el costo mínimo del subproblema S_{65} es 38.09. Más aún, como la primera suma fue la menor, se sabe que para alcanzar este valor es necesario utilizar los subproblemas S_{62} y S_{04} , esto es, seleccionar la cuerda (v_0, v_3) y resolver S_{64} como se pueda; la cuerda (v_1, v_3) es la elección preferida para este subproblema. $\square$

Existe un truco útil para llenar la tabla de la figura 10.11 de acuerdo con la fórmula (10.5). Cada término de la operación *min* de (10.5) requiere un par de valores. El primer par, para $k = 1$, puede encontrarse en la tabla a) en la parte «inferior» (la fila de $s = 2$) † de la columna del elemento que se está calculando, y b) justo abajo y a la derecha †† del elemento calculado. El segundo par está a) junto a la parte inferior de la columna, y b) dos posiciones más abajo y a la derecha. La figura 10.12 muestra las dos líneas de posiciones que se siguen para obtener todos los pares de valores que es necesario considerar simultáneamente. El patrón —subir por la columna y bajar por la diagonal— es común para el llenado de tablas durante la programación dinámica.

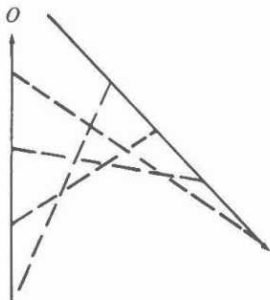


Fig. 10.12. Patrón de seguimiento de la tabla para calcular un elemento.

Para encontrar soluciones a partir de la tabla

Aunque la figura 10.11 ofrece el costo de la triangulación mínima, no proporciona de inmediato la triangulación misma. Para cada posición se necesita el valor de k que produjo el mínimo en (10.5). Luego se puede deducir que la solución consta de las cuerdas (v_i, v_{i+k}) y (v_{i+k}, v_{i+s-1}) (a menos que una de ellas no sea cuerda, porque $k = 1$ o $k = s - 2$), más las cuerdas que estén implicadas por las soluciones a $S_{i,k+1}$ y $S_{i+k,s-k}$. Al calcular un elemento de la tabla, es útil incluir el valor de k que brinde la mejor solución.

Ejemplo 10.3. En la figura 10.11, el valor C_{07} , que representa la solución al problema completo de la figura 10.8, procede de los términos de $k = 5$ en (10.5). Esto es,

† Recuérdese que la tabla de la figura 10.11 tiene filas de ceros por debajo de las que se muestran.

†† Por «a la derecha» se quiere decir que la tabla se lee en forma circular. Así, si uno se encuentra en la columna de más a la derecha, la columna «a la derecha» es la que está más a la izquierda.

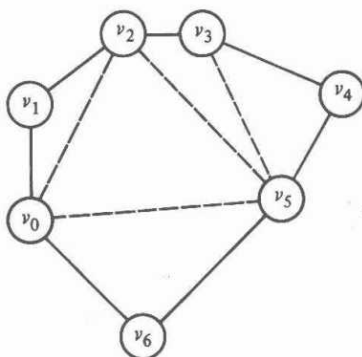


Fig. 10.13. Triangulación de costo mínimo.

el problema S_{07} se divide en S_{06} y S_{52} ; el primero es el problema con seis vértices $v_0, v_1, \dots, v_5$, y el último es un «problema» trivial de costo 0. Así, se introduce la cuerda (v_0, v_5) de costo 22.09 y se debe resolver S_{06} .

El costo mínimo de C_{06} procede de los términos para $k = 2$ en (10.5), en tanto el problema S_{06} se divide en S_{03} y S_{24} . El primero es el triángulo con vértices v_0, v_1 y v_2 , mientras que el último es un cuadrilátero definido por v_2, v_3, v_4 y v_5 . S_{03} no necesita ser resuelto, pero S_{24} sí, y deben incluirse los costos de las cuerdas (v_0, v_2) y (v_2, v_5) , los cuales son 17.89 y 19.80, respectivamente. El valor mínimo para C_{24} se supone cuando $k = 1$ en (10.5), dando los subproblemas C_{22} y C_{33} , que tienen tamaño menor o igual que tres y, por tanto, costo 0. Se introduce la cuerda (v_3, v_5) , con un costo de 15.65. □

10.3 Algoritmos ávidos

Considérese el problema de dar un cambio. Supónganse monedas de 25 ¢ (un cuarto) 10 ¢ (un décimo), 5 ¢ (un vigésimo) y 1 ¢ (un centavo), y que se desea dar un cambio de 63 ¢. Sin pensar, se convierte esta cantidad a dos cuartos, un décimo y tres centavos. No sólo se determinó rápidamente una lista de monedas con el valor correcto, sino que se produjo la lista más corta de monedas con ese valor.

El algoritmo empleado probablemente fue seleccionar la moneda mayor cuyo valor no excedía de 63 ¢ (un cuarto), agregarla a la lista y sustraer su valor de 63, quedando 38 ¢. De nuevo, se seleccionó la moneda más grande cuyo valor no fuera mayor de 38 ¢ (otro cuarto) y se añadió a la lista, y así sucesivamente.

Este método de dar cambio es un *algoritmo ávido*. En cualquier etapa individual, un algoritmo ávido selecciona la opción que sea «localmente optimal» en algún sentido particular. Obsérvese que el algoritmo ávido para dar cambio produce una solución optimal general debido a las propiedades de las monedas. Si las monedas tuvieran valores 1 ¢, 5 ¢ y 11 ¢ y se deseara dar un cambio de 15 ¢, el

algoritmo ávido seleccionaría primero la moneda de 11 ¢ y después cuatro de 1 ¢, para un total de cinco monedas. Sin embargo, tres monedas de 5 ¢ bastan.

Ya se han visto varios algoritmos ávidos, como el algoritmo del camino más corto de Dijkstra y el algoritmo de árboles abarcadores de costo mínimo de Kruskal. El algoritmo de Dijkstra es «ávido» en el sentido de que siempre escoge el vértice más cercano al origen de entre aquellos cuyo camino más corto todavía se desconoce. El algoritmo de Kruskal también es «ávido»: de las aristas restantes, elige la más corta de entre aquellas que no crean un ciclo.

Es necesario subrayar el hecho de que no todos los enfoques ávidos llegan a dar mejor resultado. Como sucede en la vida real, una estrategia ávida puede dar un buen resultado durante un tiempo, pero el resultado general puede ser pobre. Como ejemplo, se considera lo que sucede al permitir aristas ponderadas negativas en los algoritmos de Dijkstra y de Kruskal. Resulta que el algoritmo de árboles abarcadores de Kruskal no se ve afectado y continuará produciendo el árbol de costo mínimo, pero el algoritmo de Dijkstra en algunos casos no produce los caminos más cortos.

Ejemplo 10.4. En la figura 10.14 se presenta un grafo con una arista entre b y c cuyo costo es negativo. Al aplicar el algoritmo de Dijkstra con origen s , primero se descubrirá, correctamente, que el camino mínimo hacia a tiene longitud 1. Ahora, considerando sólo las aristas de s o a a b o c , se espera que b tenga el camino más corto desde s , es decir, $s \rightarrow a \rightarrow b$, de longitud 3. Así, se descubre que c tiene un camino más corto desde s de longitud 1.

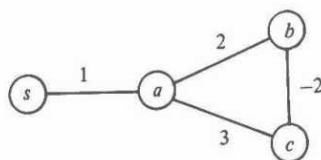


Fig. 10.14. Grafo con una arista ponderada negativa.

Sin embargo, esta selección «ávida» de b antes que c fue errónea desde un punto de vista general. Dado que el camino $s \rightarrow a \rightarrow c \rightarrow b$ tiene una longitud de sólo 2, la distancia mínima de s para b fue errónea †. □

Algoritmos ávidos como procedimientos heurísticos

Para algunos problemas, no existen algoritmos ávidos conocidos que produzcan soluciones optimales; con todo, existen algunos que pueden llegar a producir soluciones «buenas» con una alta probabilidad. A menudo, una solución semióptimal

† De hecho, se debe tener cuidado con el significado de «camino más corto» cuando existen aristas negativas. Si se permiten ciclos con costo negativo, entonces se pueden recorrer repetidamente ese ciclo para alcanzar distancias negativas arbitrariamente grandes, por lo que se desea ceñirse a caminos acíclicos.

con un costo de un pequeño porcentaje sobre la optimal es muy satisfactoria. En esos casos, un algoritmo ávido con frecuencia brinda la forma más rápida para llegar a una solución «buena». De hecho, si el problema es tal que la única forma de alcanzar una solución optimal es mediante el uso de una técnica de búsqueda exhaustiva, un algoritmo ávido u otro procedimiento heurístico para llegar a una buena solución, pero no necesariamente optimal, puede ser la única opción real.

Ejemplo 10.5. Se presenta un conocido problema donde los únicos algoritmos conocidos que producen soluciones optimales son del tipo «intentar todas las posibilidades» y pueden tener tiempos de ejecución con entradas de tamaño exponencial. El problema, llamado *problema del agente viajero*, o *PAV*, es encontrar, en un grafo no dirigido con pesos en las aristas, un *recorrido* (un ciclo simple que incluya todos los vértices) donde la suma de todos los pesos de las aristas sea un mínimo. Un recorrido se suele denominar ciclo de Hamilton (o ciclo hamiltoniano).

La figura 10.15(a) muestra un ejemplo del problema del agente viajero, un grafo con seis vértices (a veces llamados «ciudades»). Se dan las coordenadas de cada vértice, y se toma el peso de cada arista como su longitud. Obsérvese que, por convención con el PAV, se supone que todas las aristas existen, esto es, el grafo es completo. En ejemplos más generales, donde los pesos de las aristas no estén basados en la distancia euclidiana, se puede encontrar un peso infinito sobre aristas que en realidad no están presentes.

Las figuras 10.15(b) a (e) muestran cuatro recorridos de las seis «ciudades» de la figura 10.15(a). El lector puede preguntarse cuál de los cuatro es el optimal, si lo es alguno. Las longitudes de los cuatro recorridos son 50.00, 49.73, 48.39 y 49.78, respectivamente; (d) es el más corto de todos los recorridos posibles.

El PAV tiene varias aplicaciones prácticas. Como su nombre indica, puede usarse para planear la ruta de una persona que debe visitar varios sitios y re-

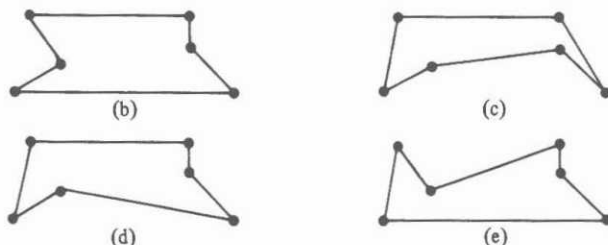
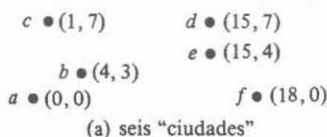


Fig. 10.15. Ejemplo del problema del agente viajero.

gresar al punto inicial. Por ejemplo, el PAV se ha usado para seleccionar la ruta de los recolectores de dinero de teléfonos públicos. Los vértices son los teléfonos y la oficina de cobro. El costo de cada arista es el tiempo del viaje entre los dos sitios en cuestión.

Otra aplicación del PAV es la resolución del *problema del recorrido del caballo* para encontrar una secuencia de movimientos de modo que el caballo pueda visitar cada cuadro del tablero de ajedrez sólo una vez y regrese al punto de partida. Se dan como vértices los cuadros del tablero de ajedrez y las aristas entre dos cuadros que son un movimiento del caballo con peso 0; las demás aristas tienen peso infinito. Un recorrido optimal tiene peso 0 y debe ser un recorrido del caballo. Sorprendentemente, los procedimientos heurísticos buenos para el PAV no tienen dificultad en encontrar los recorridos del caballo, pero encontrar uno «a mano» es un reto.

El algoritmo ávido para el PAV que se tiene en mente es una variante del algoritmo de Kruskal. Como en dicho algoritmo, primero se considerarán las aristas más cortas. En el algoritmo de Kruskal, se acepta una arista en su turno si no forma un ciclo con las que ya han sido tomadas; en caso de no cumplir esto, rechaza la arista. Para el PAV, el criterio de aceptación es que una arista sometida a consideración, junto con las que ya se han seleccionado,

1. no haga que un vértice tenga grado 3 o mayor, y
2. no forme un ciclo, a menos que el número de aristas seleccionadas sea igual al número de vértices del problema.

Las colecciones de aristas seleccionadas con ese criterio, formarán una colección de caminos no conexos, hasta el último paso, cuando el único camino existente esté cerrado y forme un recorrido.

En la figura 10.15(a), se puede tomar primero la arista (d, e) , puesto que es la más corta, con longitud 3. Después, las aristas (b, c) , (a, b) y (e, f) , que tienen longitud 5. No importa en qué orden se tomen, todas cumplen con las condiciones de selección, y han de seleccionarse si se quiere seguir el enfoque ávido. La siguiente arista más corta es (a, c) , con longitud 7.08. Sin embargo, esta arista puede formar un ciclo con (a, b) y (b, c) , por lo que se rechaza. La arista (d, f) es la siguiente que se rechaza por motivos similares. La arista (b, e) es la siguiente a considerar, pero debe rechazarse porque puede aumentar los grados de los vértices b y e a 3, y nunca formaría un recorrido con lo seleccionado. De modo similar, se rechaza (b, d) . A continuación se considera (c, d) , que sí se acepta. Ahora existe un camino, $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow f$, y por último se acepta (a, f) para completar el recorrido. El recorrido resultante está en la figura 10.15(b), y es el cuarto mejor de todos los posibles, pero es más costoso que el optimal en menos del 4%. □

10.4 Método de retroceso (*backtracking*)

Algunas veces surge el problema de encontrar una solución optimal a un subproblema, pero sucede que no existe una teoría que pueda aplicarse para ayudar a encontrar lo óptimo, si no es recurriendo a una búsqueda exhaustiva. Esta sección se de-

dica a una técnica de búsqueda exhaustiva sistemática conocida como método de retroceso (*backtracking*) y una técnica llamada poda alfa-beta, que suele reducir mucho la búsqueda.

Considérese un juego como el ajedrez, damas o tres en raya (gato), donde hay dos jugadores. Los jugadores alternan las jugadas, y el estado del juego puede representarse por una posición del tablero. Se hace la suposición de que hay un número finito de posiciones del tablero y que el juego tiene algún tipo de regla para asegurar la terminación. Con cada uno de esos juegos se asocia un árbol llamado *árbol de juego*. Cada nodo del árbol representa una posición en el tablero, y se asocia la raíz con la posición de partida. Si la posición del tablero x se asocia con el nodo n , los hijos de n corresponderán al conjunto de movimientos posibles desde la posición x del tablero, y cada hijo se asociará con la posición resultante. Por ejemplo, la figura 10.16 muestra parte del árbol del juego de tres en raya.

Las hojas del árbol corresponden a las posiciones del tablero donde ya no existen movimientos posibles, porque uno de los jugadores ha ganado o porque todos los cuadros están ocupados y se produjo un empate. Se asocia un valor a cada nodo del árbol, y primero se asignan valores a las hojas. Si el juego es tres en raya, una hoja tendrá un valor -1 , 0 ó 1 , dependiendo de si la posición del tablero corresponde a una derrota, empate o victoria para el jugador 1 (jugando con X).

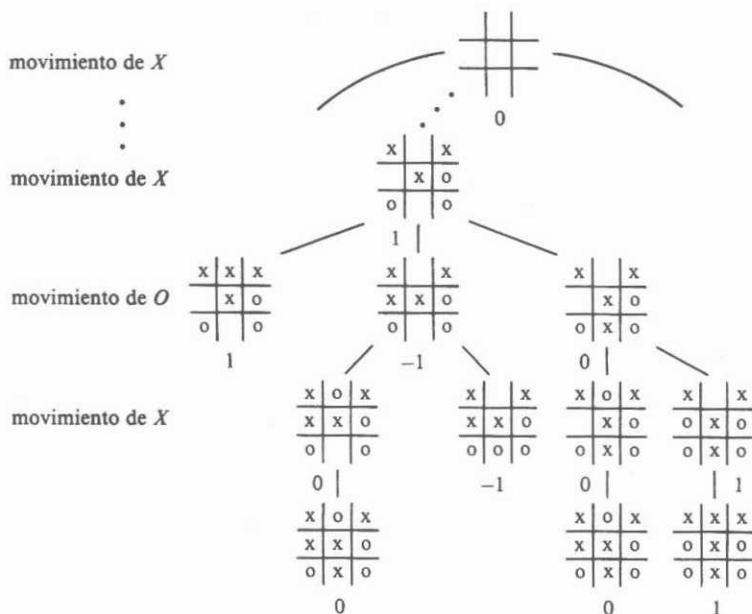


Fig. 10.16. Parte del árbol del juego de tres en raya.

Los valores se propagan hacia arriba en el árbol de acuerdo con la siguiente regla: si un nodo corresponde a una posición del tablero donde hay movimiento del jugador 1, el valor será el máximo de los valores de los hijos de ese nodo. Esto es, se supone que el jugador 1 hará el movimiento más favorable para él, o sea, el que produce el resultado con mayor valor. Si el nodo corresponde al movimiento del jugador 2, entonces el valor será el mínimo de los valores de los hijos. Es decir, se presume que el jugador 2 hará su movimiento más favorable, consiguiendo, de ser posible, una derrota para el jugador 1, o bien un empate como siguiente preferencia.

Ejemplo 10.6. Los valores de los tableros se encuentran en la figura 10.16. La hojas que significan triunfos para O tienen el valor -1 , los empates toman el 0 , y los triunfos de X tienen $+1$. En seguida se procede hacia arriba en el árbol. En el nivel 8, donde sólo queda un cuadro vacío, y es un movimiento de X , los valores de los tableros no resueltos son el «máximo» del hijo en el nivel 9.

En el nivel 7, correspondiente a un movimiento de O , hay dos opciones y se toma como valor para un nodo interno el mínimo de los valores de sus hijos. El tablero de la extrema izquierda, mostrado en el nivel 7, es una hoja y tiene un valor 1 , porque corresponde a un triunfo de X . El segundo tablero del mismo nivel tiene un valor mín $(0, -1) = -1$, mientras que el tercero tiene valor mín $(0, 1) = 0$. El único tablero mostrado en el nivel 6, que corresponde a un movimiento de X , tiene un valor máx $(1, -1, 0) = 1$, lo que significa que existe una selección que X puede hacer para ganar; en tal caso, el triunfo es inmediato. $\square$

Nótese que si la raíz tiene valor 1 , el jugador 1 tiene una estrategia ganadora; cualquiera que sea su turno, tiene la garantía de seleccionar un movimiento que le dé una posición del tablero con valor 1 . Cuando el turno corresponde al jugador 2, no tiene más opción que seleccionar un movimiento que deje una posición del tablero con valor 1 , lo cual significa una derrota para él. El hecho de que se suponga que el juego termina, garantiza que el primer jugador podrá ganar. Si la raíz tiene valor 0 , como sucede en el juego tres en raya, ningún jugador tiene una estrategia ganadora, pero por lo menos puede garantizarse un empate si juega lo mejor posible. Si la raíz tiene valor -1 , entonces el jugador 2 tiene una estrategia ganadora.

Funciones de utilidad

La idea de un árbol de juego, donde los nodos tienen valores -1 , 0 y 1 , puede generalizarse a árboles donde a las hojas se les da un número cualquiera (llamado la *utilidad*) como valor, y se aplican las mismas reglas de evaluación de nodos interiores: toma el máximo de los hijos en los niveles donde corresponda un movimiento del jugador 1, y el mínimo de los hijos en los niveles donde mueve el jugador 2.

Como ejemplo de donde son provechosas las utilidades generales, considérese un juego complejo como el ajedrez, donde el árbol de juego, aunque finito, es tan inmenso que la evaluación de abajo arriba no es posible. Los programas de ajedrez operan construyendo el árbol de juego con ese tablero como raíz para cada posición del tablero desde la cual se debe mover extendiéndose hacia abajo por varios niveles; el número exacto de niveles depende de la velocidad con que el computador puede tra-

bajar. Puesto que la mayor parte de las hojas del árbol son ambiguas, no indican triunfos, derrotas, ni empates, cada programa usa una función de las posiciones del tablero que intenta evaluar la probabilidad de que el computador gane desde esa posición. Por ejemplo, la diferencia entre las piezas afecta en gran medida tal estimación, al igual que algunos factores, como el poder defensivo alrededor de los reyes. Al usar esta función de utilidad, el computador puede estimar la probabilidad de un triunfo después de hacer cada uno de sus siguientes movimientos posibles en el supuesto de que se hará la mejor jugada siguiente de cada lado, y se escogerá el movimiento de mayor utilidad †.

Realización de la búsqueda con retroceso

Supóngase que se proporcionan las reglas de un juego ††, esto es, sus movimientos válidos y reglas de terminación. Se desea construir su árbol de juego y evaluar la raíz, en la manera más obvia, y después visitar los nodos en orden final. El recorrido en orden final asegura que se visita un nodo interior n después de sus hijos, y entonces es posible evaluar n , tomando el mínimo o el máximo de los valores de sus hijos, lo que sea más apropiado.

El espacio de almacenamiento del árbol puede ser demasiado grande, pero con cuidado nunca se necesitará almacenar más de un camino, desde la raíz hasta un nodo, en cualquier momento. En la figura 10.17 se presenta el esbozo de un programa recursivo que representa el camino en el árbol por medio de la secuencia de las llamadas a procedimientos activos en cualquier instante. Ese programa supone lo siguiente:

1. Las utilidades son números reales en un intervalo limitado, por ejemplo -1 a $+1$.
2. La constante ∞ es mayor que cualquier utilidad positiva y su negativo es menor que cualquier utilidad negativa.
3. El tipo `tipo_mod` se declara

```
type
    tipo_mod = (MIN, MAX)
```

4. Existe un tipo `tipo_tablero` declarado de alguna forma adecuada para la representación de posiciones en el tablero.

† Inicialmente, otras cosas que hacen los programas para jugar al ajedrez son las siguientes.

1. Usar técnicas heurísticas para eliminar la consideración de ciertos movimientos que tal vez no serán buenos. Esto ayuda a expandir el árbol a más niveles en un tiempo dado.
2. Expandir «cadenas de captura» que son secuencias de movimientos de captura más allá del último nivel al que se expande el árbol normalmente. Esto ayuda a estimar con mayor exactitud la fuerza material relativa de las posiciones.
3. Podar el árbol de búsqueda por medio de la técnica alfa-beta, como se analizará más adelante en esta sección.

†† Esto no implica que de esta forma sólo se puedan resolver «juegos». Como se verá en ejemplos posteriores, el «juego» puede representar en realidad la solución de un problema práctico.

5. Existe una función UTILIDAD que calcula la utilidad de cualquier tablero que sea una *hoja* (esto es, situación de triunfo, derrota o empate).

```

function busca ( B: tipo_tablero; modo: tipo_modo ) : real;
    { evalúa la utilidad del tablero B, en el supuesto
      de que es el movimiento del jugador 1 si modo = MAX, y es
      el movimiento del jugador 2 si modo = MIN. Devuelve la utilidad }
    var
        C: tipo_tablero; { un hijo del tablero B }
        valor: real; { valor mínimo o máximo temporal }
    begin
        (1)      if B es una hoja then
        (2)          return (utilidad(B))
        else begin
            { asigna el valor inicial mínimo o máximo de los hijos }
            (3)      if modo = MAX then
            (4)          valor := -∞
            else
                (5)      valor := ∞;
            (6)      for cada hijo C del tablero B do
                (7)          if modo = MAX then
                (8)              valor := máx(valor, busca(C, MIN))
                else
                    (9)      valor := mín(valor, busca(C, MAX));
            (10)     return (valor)
        end
    end; { busca }
  
```

Fig. 10.17. Programa de búsqueda recursiva con el método de retroceso.

Otra implantación a considerar es por medio de un programa no recursivo que contenga una pila de tableros correspondientes a la secuencia de llamadas activas de *busca*. Las técnicas estudiadas en la sección 2.6 pueden usarse para construir dicho programa.

Poda alfa-beta

Hay una observación simple que permite dejar de considerar buena parte de un árbol de juego típico. Partiendo de la figura 10.17, el ciclo **for** de la línea (6) puede saltar ciertos hijos, con frecuencia muchos. Supóngase que se tiene un nodo n , como en la figura 10.18, y ya se ha determinado que c_1 , el primero de los hijos de n , tiene valor 20. Como n es un nodo máx, se sabe que su valor es por lo menos 20. Ahora, supóngase que al continuar con la búsqueda se observa que d , un hijo de c_2 , tiene valor 15. Como c_2 , otro hijo de n , es un nodo mín, se sabe que el valor de c_2 no pue-

de exceder de 15. Así, cualquiera que sea el valor c_2 no afecta al valor de n ni de cualquier padre de n .

Así, en la situación de la figura 10.18, es posible ignorar la consideración de los hijos de c_2 que aún no se han examinado. Las reglas generales para pasar por alto o «podar» nodos implican la noción de valores tentativos y finales para los nodos. El valor *final* es el que se ha denominado hasta ahora simplemente «valor». Un valor

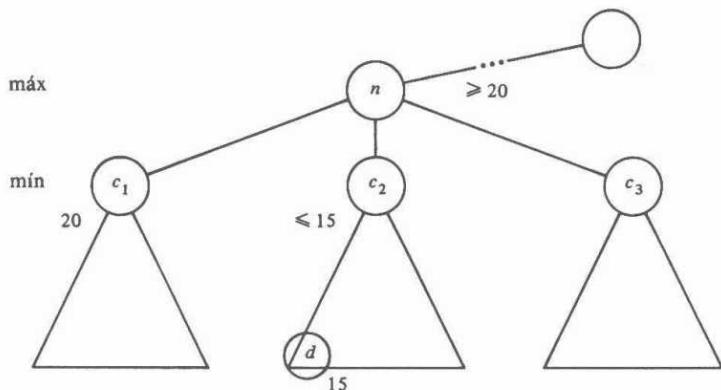


Fig. 10.18. Poda de los hijos de un nodo.

tentativo es una cota superior al valor del nodo mín, o una cota inferior al valor del nodo máx. Las reglas para el cálculo de los valores finales y tentativos son las siguientes.

1. Si todos los hijos de un nodo n se consideraron o se podaron, conviértase el valor tentativo de n en final.
2. Si un nodo máx n tiene valor tentativo v_1 y un hijo con valor final v_2 , entonces el valor tentativo de n se hace $\max(v_1, v_2)$. Si n es un nodo mín, se asigna su valor tentativo a $\min(v_1, v_2)$.
3. Si p es un nodo mín con padre q (un nodo máx), y p y q tienen valores tentativos v_1 y v_2 , respectivamente, con $v_1 \leq v_2$, entonces se pueden podar todos los hijos no considerados de p . También se pueden podar los hijos no considerados de p si p es un nodo máx (y por tanto, q es un nodo mín) y $v_2 \leq v_1$.

Ejemplo 10.7. Considérese el árbol de la figura 10.19. Suponiendo los valores mostrados en las hojas, se desea calcular el valor de la raíz; se empieza con un recorrido en orden final. Después de alcanzar el nodo D , por la regla (2) se asigna un valor tentativo de 2 al nodo C , que es el valor final de D . Entonces se explora E y se vuelve a C y después a B . Por la regla (1), el valor final de C se fija en 2 y el valor de B se hace tentativamente 2. La búsqueda descende hasta G y después regresa a F . El valor de F se hace tentativamente 6. Por la regla (3), con p y q iguales a F y B , respec-

tivamente, se puede podar H . Esto es, no se necesita explorar el nodo H , ya que el valor tentativo de F nunca puede decrecer y ya es mayor que el valor de B , que no puede aumentar.

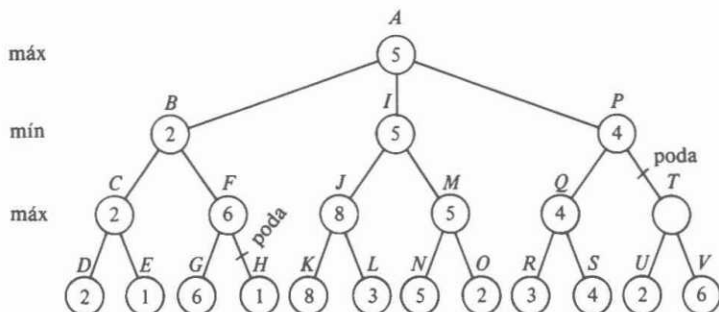


Fig. 10.19. Árbol de juego.

Continuando con el ejemplo, A tiene asignado un valor tentativo de 2 y la búsqueda prosigue hasta K . J tiene asignado un valor tentativo de 8. L no determina el valor del nodo J . I recibe un valor tentativo de 8. La búsqueda sigue hacia abajo hasta N , y a M se le asigna un valor tentativo de 5. El nodo O debe ser explorado, ya que 5, el valor tentativo de M , es menor que el valor tentativo de I . Se revisan los valores tentativos de I y A y la búsqueda continúa hasta R . Finalmente, se exploran R y S , y P recibirá un valor tentativo de 4. No es necesario buscar en T ni más abajo, ya que eso sólo disminuiría el valor de P , que ya es demasiado pequeño para afectar al valor de A . □

Búsqueda de ramificación y acotamiento

Los juegos no son la única clase de «problemas» que pueden resolverse explorando exhaustivamente un árbol completo de posibilidades. Hay muchos problemas que exigen encontrar una configuración mínima o máxima de alguna clase que son susceptibles de resolución por medio de la búsqueda de retroceso de un árbol de todas las posibilidades. Los nodos del árbol pueden considerarse como conjuntos de configuraciones y los hijos de un nodo n representan cada uno un subconjunto de las configuraciones que n representa. Finalmente, cada hoja representa configuraciones sencillas o soluciones al problema, y se puede evaluar cada una de las configuraciones con el fin de saber si es la mejor solución encontrada hasta ese momento.

Si se es hábil en el diseño de la búsqueda, los hijos de un nodo representarán muchas menos configuraciones que el nodo mismo, así que no es necesario profundizar mucho para alcanzar las hojas. A fin de evitar que esta noción de búsqueda parezca muy vaga, he aquí un ejemplo concreto.

Ejemplo 10.8. Recuérdese de la sección anterior el problema del agente viajero, en el que se dio un «algoritmo ávido» para encontrar un buen recorrido, aunque no necesariamente el óptimo. Ahora se analiza cómo encontrar el recorrido óptimo por medio de la consideración sistemática de todos los recorridos. Una forma es tener en cuenta todas las permutaciones de los nodos y evaluar los recorridos que visitan los nodos en ese orden, recordando el mejor encontrado hasta el momento. El tiempo para esta aproximación es $O(n!)$ en un grafo con n nodos, ya que se deben considerar $(n-1)!$ permutaciones distintas †, y cada permutación toma un tiempo $O(n)$.

Se presentará un enfoque un poco distinto que no es mejor que el anterior en el peor caso, pero en el promedio, cuando se acopla con una técnica llamada «ramificación y acotamiento» —que se estudiará a continuación— produce la respuesta mucho más rápido que el método de «intentar con todas las permutaciones». Se empieza construyendo un árbol, con una raíz que representa todos los recorridos. Los recorridos son lo que se denominó «configuraciones» en el material preliminar. Cada nodo tiene dos hijos, y los recorridos que representa un nodo están divididos por esos dos hijos en dos grupos: los que tienen una arista particular y los que no. Por ejemplo, la figura 10.20 muestra porciones del árbol para el caso del PAV de la figura 10.15(a)

En la figura 10.20 se han considerado las aristas en orden lexicográfico (a, b) , (a, c) , ..., (a, f) , (b, c) , ..., (b, f) , (c, d) , y así sucesivamente. Claro está que se puede tomar cualquier otro orden. Obsérvese que no todos los nodos del árbol tienen dos hijos. Se pueden eliminar algunos hijos, porque las aristas seleccionadas no forman parte de un recorrido. Así, no hay nodo para los «recorridos que contienen (a, b) , (a, c) y (a, d) », porque a tendría grado 3 y el resultado no sería un recorrido. De forma semejante, al ir descendiendo en el árbol, se eliminan algunos nodos, porque alguna ciudad puede tener grado menor que 2. Por ejemplo, no habrá nodos para recorridos sin (a, b) , (a, c) , (a, d) o (a, e) . □

Procedimientos heurísticos necesarios para ramificación y acotamiento

Al utilizar ideas similares a las de la poda alfa-beta, se eliminan muchos más nodos del árbol de búsqueda que los mostrados en el ejemplo 10.8. Supóngase, para ser específicos, que el problema es minimizar alguna función, como el costo de un recorrido en el PAV. Supóngase, también, que se tiene un método para obtener una cota inferior en el costo de cualquier solución entre las del conjunto de soluciones representado por algún nodo n . Si la mejor solución encontrada hasta ahora cuesta menos que la cota inferior para el nodo n , no es necesario explorar ninguno de los nodos por debajo de n .

Ejemplo 10.9. Se revisará una forma de alcanzar una cota inferior en ciertos conjuntos de soluciones para el PAV, los representados por nodos en un árbol de solu-

† Obsérvese que no es necesario considerar todas las $n!$ permutaciones, ya que el punto de partida de un recorrido no importa. Por tanto, se consideran sólo aquellas permutaciones que empiecen con 1.

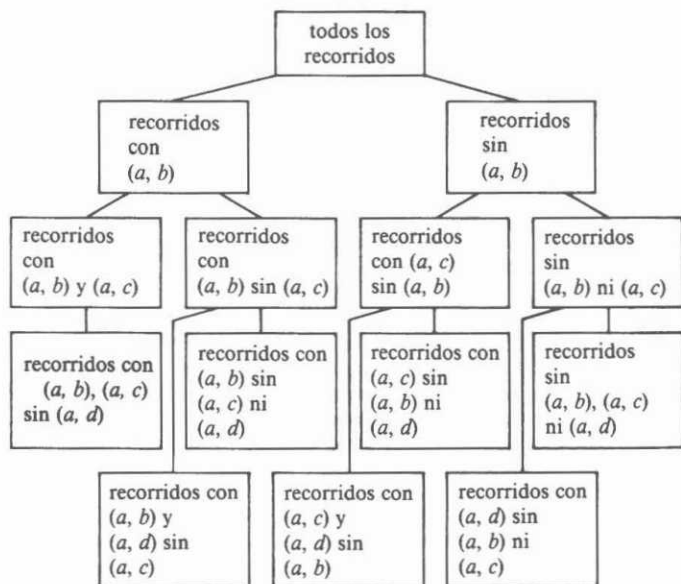


Fig. 10.20. Principio de un árbol solución para un caso del PAV.

ciones, como se sugirió en la figura 10.20. Primero, supóngase que se desea una cota inferior para todas las soluciones de un caso dado del PAV. Obsérvese que el costo de cualquier recorrido puede expresarse como la semisuma, sobre todos los nodos n , del costo de las dos aristas del recorrido adyacentes a n . Esta observación da lugar a la siguiente regla: la suma de las dos aristas de recorrido adyacentes al nodo n no es menor que la suma de las dos aristas de costo menor adyacentes a n . Así, ningún recorrido puede costar menos que la semisuma sobre todos los nodos n de las dos aristas de menor costo que inciden sobre n .

Por ejemplo, considérese el caso del PAV que se muestra en la figura 10.21. A diferencia del caso de la figura 10.15, la medida de la distancia de las aristas no es euclidiana, es decir, no tiene relación con la distancia en el plano entre las ciudades que conecta. Tal medida del costo puede ser el tiempo o la tarifa del viaje, por ejemplo. En este caso, las aristas de menor costo adyacentes al nodo a son (a, d) y (a, b) , con un costo total de 5. Para el nodo b , se encuentran (a, b) y (b, e) con un costo total de 6. De igual forma, las dos aristas de menor costo adyacentes a c , d y e , totalizan 8, 7 y 9, respectivamente. Así pues, la cota inferior del costo de un recorrido es $(5+6+8+7+9)/2 = 17.5$.

Ahora, supóngase que se pide una cota inferior en el costo de un subconjunto de recorridos definidos por algún nodo en el árbol de búsqueda. Si el árbol de búsqueda se construye como en el ejemplo 10.8, cada nodo representa recorridos definidos

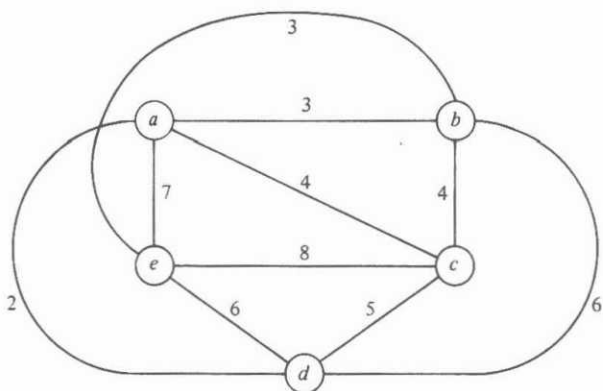


Fig. 10.21. Un caso del PAV.

por un conjunto de aristas que deben estar en el recorrido y un conjunto de aristas que tal vez no estén. Esas *restricciones* alteran las opciones para escoger las dos aristas de menor costo en cada nodo. Es evidente que una arista restringida a permanecer en cualquier recorrido debe incluirse entre las dos aristas seleccionadas, sin importar si son del menor o segundo menor costo o no †. Asimismo, una arista restringida a estar fuera no puede seleccionarse, aunque su costo sea menor.

Así, si existe la restricción de incluir la arista (a, e) y excluir (b, c) , las dos aristas para el nodo a son (a, d) y (a, e) , con un costo total de 9. Para b , se seleccionan (a, b) y (b, e) , igual que antes, con un costo total de 6. Para c , no es posible elegir (b, c) , y se eligen (a, c) y (c, d) , con un costo total de 10. Para d , se seleccionan (a, d) y (c, d) , como antes, mientras que para e , se deben escoger (a, e) y (b, e) . La cota inferior para esas restricciones es $(9+6+9+7+10)/2 = 20.5$. □

Ahora se construye el árbol de búsqueda a partir de lo sugerido en el ejemplo 10.8. Se consideran las aristas en orden lexicográfico, como en ese ejemplo. Por cada *ramificación*, al considerar los dos hijos de un nodo, se intenta inferir decisiones adicionales sobre las aristas que deben incluirse o excluirse de los recorridos representados por esos nodos. Las reglas utilizadas para esas inferencias son:

1. Si la exclusión de una arista (x, y) hiciera imposible para x o y tener dos aristas adyacentes en el recorrido, entonces debe incluirse (x, y) .
2. Si incluir (x, y) hace que x o y tengan más de dos aristas adyacentes en el recorrido, o que se complete un ciclo que no sea recorrido con las aristas ya incluidas, entonces debe excluirse (x, y) .

† Se verá que las reglas para la construcción del árbol de búsqueda eliminan cualquier conjunto de restricciones que no puedan producir un recorrido, por ejemplo, porque se requieran tres aristas adyacentes a un nodo para formar parte del recorrido.

Al ramificar, después de hacer todas las inferencias posibles, se calculan las cotas inferiores de ambos hijos. Si la cota inferior de un hijo es tan grande o mayor que el recorrido de costo menor encontrado hasta ese momento, se «poda» ese hijo y no se construyen ni consideran sus descendientes. Curiosamente, hay situaciones donde la cota inferior de un nodo n es menor que la mejor solución existente encontrada, y aún así ambos hijos de n se pueden podar, porque sus cotas inferiores exceden del costo de la mejor solución encontrada hasta el momento.

Si no es posible podar ningún hijo, como regla heurística se considerará primero el hijo con la cota inferior más pequeña, con la esperanza de alcanzar más rápido una solución más económica que la mejor encontrada hasta el momento †. Después de considerar un hijo, se debe evaluar otra vez si puede podarse su hermano, ya que pudo haberse encontrado una mejor solución nueva. Para el caso de la figura 10.21, se obtiene árbol de la figura 10.22. Para interpretar los nodos de ese árbol, es útil entender que las letras mayúsculas son los nombres de los nodos del árbol de búsqueda; los números son las cotas inferiores, y se listan las restricciones aplicadas al nodo, pero no sus antecesores, escribiendo xy si la arista (x, y) debe incluirse, y $\overline{xy}$, si (x, y) debe excluirse. Obsérvese también que las restricciones introducidas en un nodo se aplican a todos sus descendientes. Así, para obtener todas las restricciones aplicables en un nodo debe seguirse el camino desde ese nodo hasta la raíz.

Por último, se hace hincapié en que, como para la búsqueda de retroceso general, se construye el árbol nodo por nodo, reteniendo sólo un camino, como en el algoritmo recursivo de la figura 10.17, o su contraparte no recursiva. Quizá se prefiera la versión no recursiva, para mantener dentro de una pila la lista de restricciones.

Ejemplo 10.10. La figura 10.22 muestra el árbol de búsqueda para el caso del PAV de la figura 10.22. Con el fin de ver cómo se construye, se parte de la raíz A de la figura 10.22. La primera arista en orden lexicográfico es (a, b) , así que se consideran los hijos B y C , correspondientes a las restricciones ab y $\overline{ab}$, respectivamente. De momento no hay una «mejor solución hasta aquí», por lo que se considerarán B y C ††. Forzar la inclusión de (a, b) no sube la cota inferior, pero excluir tal arista produce la cota 18.5, ya que las dos aristas válidas más económicas para los nodos a y b totalizan 6 y 7, respectivamente, comparadas con 5 y 6 sin restricciones. Siguiendo el procedimiento heurístico, se considerarán primero los descendientes del nodo B .

La siguiente arista en orden lexicográfico es (a, c) . Se introducen los hijos D y E correspondientes a los recorridos en que (a, c) está incluida y excluida, respectivamente. En el nodo D , se infiere que ni (a, d) ni (a, e) pueden estar en un recorrido; de otra forma, a puede tener demasiadas aristas incidentes. De acuerdo con el procedimiento heurístico, se analiza E antes que D , para ramificar sobre la arista (a, d) . Los hijos F y G se introducen con cotas inferiores 18 y 23, respectivamente. Para cada uno de esos hijos se conocen unas 3 aristas incidentes sobre a , por lo que se puede deducir algo acerca de la arista restante (a, e) .

† Una alternativa es usar un procedimiento heurístico para obtener una buena solución con las restricciones requeridas para cada hijo. Por ejemplo, se deberá poder modificar el algoritmo ávido del PAV para respetar las restricciones.

†† Se puede empezar con alguna solución encontrada heurísticamente, como la ávida, aunque eso no afecte a nuestro ejemplo. La solución ávida para la figura 10.21 tiene un costo de 21.

Considérense primero los hijos de F . La primera arista restante en orden lexicográfico es (b, c) , y si se incluye, al haber incluido ya (a, b) , no se pueden incluir (b, d) ni (b, e) . Al haber eliminado (a, e) y (b, e) , se deben tener (c, e) y (d, e) . No se puede tener (c, d) , porque entonces c y d tendrían 3 aristas incidentes. Queda, por último un recorrido (a, b, c, e, d, a) , cuyo costo es 23. Igualmente, el nodo I , donde (b, c) está excluida, puede probarse que representa sólo el recorrido (a, b, e, c, d, a) , de costo 21. Este recorrido tiene el menor costo encontrado hasta el momento.

Ahora se retrocede a E y se considera su segundo hijo, G . Pero G tiene una cota inferior de 23, la cual excede al mejor costo actual, 21. Así, se poda G . Ahora, se retrocede hasta B y se considera su otro hijo, D . La cota inferior en D es 20.5, pero debido a que los costos son enteros, se sabe que ningún recorrido representado por D tiene costo menor que 21. Dado que ya existe un recorrido tan económico, no es necesario explorar los descendientes de D y, por tanto, se poda. Ahora, se retrocede hasta A y se considera su segundo hijo, C .

En el nivel del nodo C , sólo se ha considerado la arista (a, b) . Los nodos J y K se introducen como hijos de C . J corresponde a aquellos recorridos que tienen a (a, c) , pero no a (a, b) , y su cota inferior es 18.5. K corresponde a los recorridos que no contienen (a, b) ni (a, c) , y se infiere que esos recorridos tienen a (a, d) y (a, e) . La cota inferior de K es 21 y se puede podar de inmediato, dado que ya se conoce un recorrido de menor costo.

A continuación se consideran los hijos en J , L y M , y se poda M porque su cota inferior excede del costo del mejor recorrido actual. Los hijos de L son N y P , correspondientes a recorridos que tienen (b, c) , y que excluyen a (a, b) . Considerando el grado de los nodos b y c , y recordando que las aristas seleccionadas no pueden formar un ciclo con menos de cinco ciudades, se infiere que los nodos N y P representan recorridos individuales. Uno de esos, (a, c, b, e, d, a) , tiene menor costo que cualquier otro, 19. Ya se ha explorado o podado todo el árbol y, por tanto, se ha terminado. □

10.5 Algoritmos de búsqueda local

Algunas veces, la siguiente estrategia producirá una solución optimal para un problema.

1. Empezar con una solución aleatoria.
2. Aplicar a la solución actual una transformación de un conjunto dado de transformaciones para mejorar la solución; la mejora es la nueva solución «actual».
3. Repetir hasta que ninguna transformación del conjunto pueda mejorar la solución actual.

La solución resultante puede ser optimal o no. En principio, si «el conjunto de transformaciones dado» incluye todas las que toman una solución y la reemplazan por otra, entonces no se parará hasta alcanzar la solución optimal. Pero, en ese caso, el tiempo para aplicar (2) es el mismo que el necesario para examinar todas las soluciones, y todo el enfoque no tiene mucho sentido.

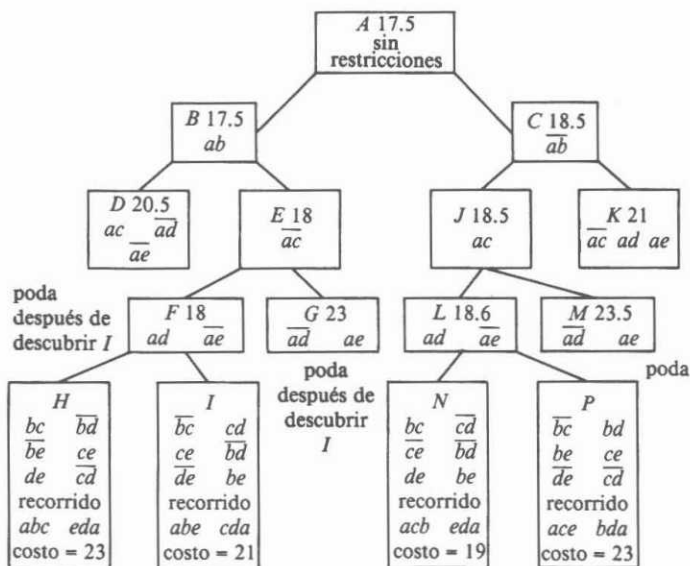


Fig. 10.22. Árbol de búsqueda para la solución del PAV.

El método tiene sentido cuando se puede restringir el conjunto de transformaciones a un conjunto pequeño, de modo que en poco tiempo se puedan considerar todas las transformaciones posibles; tal vez deben permitirse las $O(n^2)$ u $O(n^3)$ transformaciones cuando el problema es de «tamaño» n . Si el conjunto de transformaciones es pequeño, es natural considerar las soluciones, que pueden ser transformadas entre sí en un paso como «cercanas». Las transformaciones se llaman «transformaciones locales», y el método se conoce como *búsqueda local*.

Ejemplo 10.11. Un problema que se puede resolver con exactitud mediante búsqueda local es el de los árboles abarcadores minimales. Las transformaciones locales son aquellas en las que se toma alguna arista que no se encuentra en el árbol abarcador actual, se agrega al árbol, que debe producir un ciclo único, y después eliminar exactamente una arista del ciclo (probablemente la de costo mayor) para formar un árbol nuevo.

Por ejemplo, considérese el grafo de la figura 10.21. Se empieza con el árbol mostrado en la figura 10.23(a). Una transformación que se puede realizar es agregar la arista (d, e) y quitar otra del ciclo formado, el cual es (e, a, c, d, e) . Si se elimina la arista (a, e) , se reduce el costo del árbol de 20 a 19. Esa transformación puede hacerse dejando el árbol de la figura 10.23(b), al cual se intenta aplicar de nuevo una transformación de mejora, como insertar la arista (a, d) y eliminar la (c, d) del ciclo

formado. El resultado se muestra en la figura 10.23(c). Después es posible introducir (a, b) y eliminar (b, c) , como en la figura 10.23(d), y más tarde introducir (b, e) en favor de (d, e) . El árbol resultante de la figura 10.23(e) es minimal. Se puede comprobar si cada arista ausente en ese árbol tiene costo mayor que cualquier otra arista dentro del ciclo que forma. Así, ninguna transformación es aplicable a la figura 10.23(c). $\square$

El tiempo requerido por el algoritmo del ejemplo 10.11 en un grafo de n nodos y a aristas depende del número de veces que se necesite mejorar la solución. La sola prueba de que ninguna transformación es aplicable puede llevar un tiempo $O(na)$, ya que deben probarse a aristas, y cada una puede formar un ciclo de longitud cercana a n . Así, el algoritmo no es tan bueno como los algoritmos de Prim o de Kruskal, pero sirve como ejemplo de que puede obtenerse una solución optimal por búsquedas locales.

Algoritmos de aproximación por búsqueda local

Los algoritmos de búsqueda local han tenido gran efectividad como métodos heurísticos para la solución de problemas cuyas soluciones exactas requieren tiempo exponencial. Un método común de búsqueda es empezar con un número de soluciones aleatorias y aplicar las transformaciones locales a cada una, hasta alcanzar una solución *localmente optimal*, que ninguna transformación pueda mejorar. Con frecuencia se alcanzarán diferentes soluciones localmente optimales, a partir de la mayor parte o de todas las soluciones iniciales aleatorias, como se sugiere en la figura

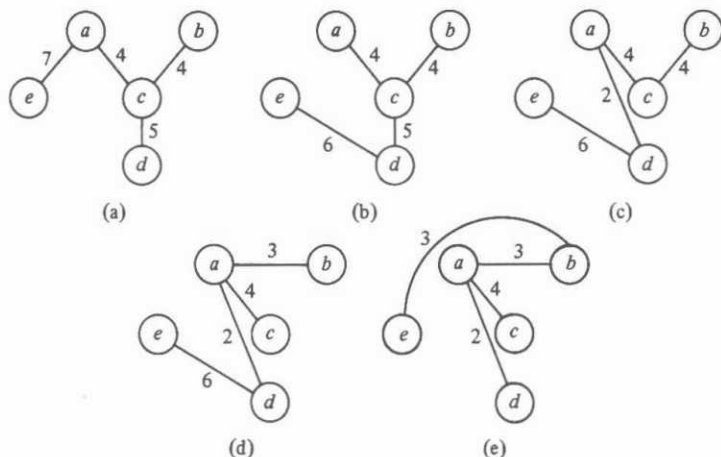


Fig. 10.23. Búsqueda local para un árbol abarcador minimal.

10.24. Si se tiene suerte, una de ellas será *globalmente optimal*, esto es, tan buena como cualquier otra solución.

En la práctica, tal vez no se pueda encontrar una solución globalmente optimal como la que se sugiere en la figura 10.24, ya que el número de soluciones localmente optimales puede ser enorme. Sin embargo, es posible escoger por lo menos aquella solución localmente optimal que tenga el menor costo entre todas las que se encontraron. Como el número de clases de transformaciones locales utilizadas para resolver varios problemas es muy grande, se cerrará la sección con dos ejemplos: el PAV y un problema simple de «colocación de paquetes».

Problema del agente viajero

El PAV es uno de los problemas en que las técnicas de búsqueda local han sido muy satisfactorias. La transformación más simple que se ha usado se conoce como «opción doble». Consiste en tomar dos aristas cualesquiera, tales como (A, B) y (C, D) de la figura 10.25, quitándolas y reconectando sus extremos para formar un recorrido nuevo. En esta figura, el recorrido nuevo va desde B hasta C en el sentido de las manecillas del reloj, después sigue por la arista (C, A) , desde A hasta D en sentido contrario al de las manecillas del reloj y finalmente por la arista (D, B) . Si la suma de las longitudes de (A, C) y (B, D) es menor que la suma de las longitudes de (A, B) y (C, D) , se tendrá un recorrido mejorado †. Obsérvese que no se pueden conectar

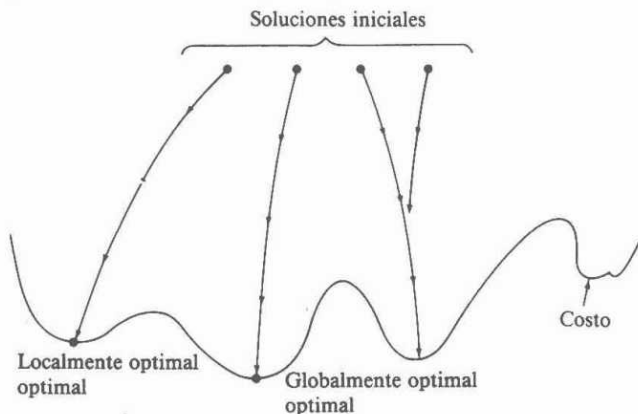


Fig. 10.24. Búsqueda local en el espacio de soluciones.

† No hay que dejarse engañar por el dibujo de la figura 10.25. Es cierto que si las longitudes de las aristas son distancias en el plano, las aristas con puntos deben ser mayores que las que reemplazan. De hecho, las opciones dobles en el plano mejoran el costo exactamente cuando las aristas eliminadas, (A, B) y (C, D) , se cruzan, mientras que las reemplazadas no lo hacen. Sin embargo, no existe motivo alguno para suponer que las distancias de la figura 10.25 sean distancias en un plano, o si lo son, puede suceder que se hayan cruzado (A, B) y (C, D) y no (A, C) y (B, D) .

A con D y B con C , pues, el resultado no sería un recorrido, sino dos ciclos disjuntos. Para encontrar un recorrido localmente optimal, se comienza con un recorrido aleatorio, y se consideran todos los pares de aristas no adyacentes, como (A, B) y (C, D) de la figura 10.25. Si el recorrido puede mejorarse reemplazando esas aristas por (A, C) y (B, D) , se hace así, y se prosigue considerando los pares de aristas que no se consideraron anteriormente. Obsérvese que las aristas introducidas (A, C) y (B, D) deben emparejarse cada una con todas las demás del recorrido, ya que pueden resultar mejoras adicionales.

Ejemplo 10.12. Reconsidérese la figura 10.21, para comenzar con el recorrido de la figura 10.26(a). Se reemplazan (a, e) y (c, d) , cuyo costo total es 12, por (a, d) y (c, e) , con un costo total de 1, como se muestra en la figura 10.26(b). Después se sustituyen (a, b) y (c, e) por (a, c) y (b, e) , dando el recorrido optimal mostrado en la figura 10.26(c). Se puede comprobar que no haya ningún par de aristas que se pueda eliminar de esta figura y ser reemplazado con ventaja por aristas cruzadas con los mismos extremos. Como un caso adicional, (b, c) y (d, e) juntas tienen el costo relativamente alto de 10. Pero (c, e) y (b, e) son peores, al costar juntas 14. $\square$

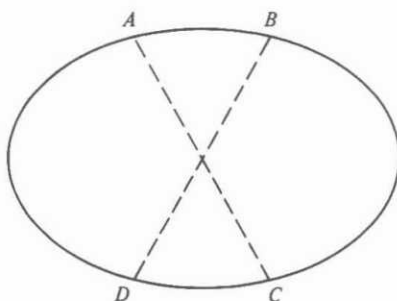


Fig. 10.25. Opción doble.

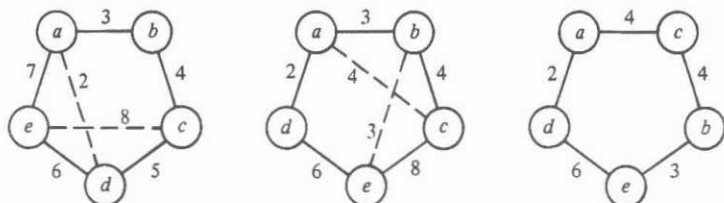


Fig. 10.26. Optimización de un caso del PAV con opción doble.

Se puede generalizar la opción doble a opción k para alguna k constante, donde se eliminan hasta k aristas y se reconectan los segmentos restantes en cualquier orden hasta producir un recorrido. Obsérvese que no se requiere que las aristas suprimidas no sean adyacentes en general, si bien para el caso de la opción doble no tenía sentido considerar la supresión de dos aristas adyacentes. Obsérvese también que para $k > 2$, existe más de una forma de conectar los segmentos. Por ejemplo, la figura 10.27 muestra el proceso general de opción triple utilizando cualquiera de los ocho conjuntos de aristas siguientes.

1. $(A, F), (D, E), (B, C)$ (como fue el recorrido)
2. $(A, F), (C, E), (D, B)$ (una opción doble)
3. $(A, E), (F, D), (B, C)$ (otra opción doble)
4. $(A, E), (F, C), (B, D)$ (una opción triple real)
5. $(A, D), (C, E), (B, F)$ (otra opción triple real)
6. $(A, D), (C, F), (B, E)$ (otra opción triple real)
7. $(A, C), (D, E), (B, F)$ (una opción doble)
8. $(A, C), (D, F), (B, E)$ (una opción triple)

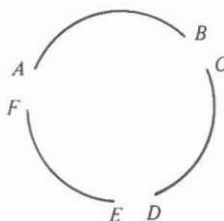


Fig. 10.27. Segmentos de un recorrido después de la supresión de tres aristas.

Es fácil verificar que, para una k fija, el número de diferentes transformaciones con opción k que se necesita considerar, si existen n vértices, es $O(n^k)$. Por ejemplo, el número exacto es $n(n-3)/2$ para $k = 2$. Sin embargo, el tiempo requerido para obtener un recorrido localmente óptimo puede ser mucho mayor que esto, ya que se pudieron haber realizado muchas transformaciones locales antes de alcanzar un recorrido localmente óptimo, y cada transformación de mejora introduce aristas nuevas que pueden participar en transformaciones posteriores que mejoran aún más el recorrido fijado. Lin y Kernighan [1973] han encontrado que la opción de profundidad variable es un método muy poderoso en la práctica, y tiene una buena oportunidad de obtener el recorrido óptimo en problemas de ciudades entre 40 y 100.

Colocación de paquetes

El problema de la *colocación unidimensional de paquetes* puede plantearse como sigue. Se tiene un grafo no dirigido, cuyos vértices se denominan «paquetes». Las aristas están etiquetadas por sus «pesos», y el peso $w(a, b)$ de la arista (a, b) es el nú-

mero de hilos existente entre los paquetes a y b . El problema es ordenar los vértices $p_1, p_2, \dots, p_n$ de manera que la suma de $|i - j| w(p_i, p_j)$ en todos los pares i y j se minimice. Esto es, se desea minimizar la suma de las longitudes de los hilos necesarios para conectar todos los paquetes con el número requerido de hilos.

Al problema de la colocación de paquetes se le han dado muchas aplicaciones. Por ejemplo, los «paquetes» pueden ser tarjetas lógicas en un fichero, y el peso de una interconexión entre tarjetas es el número de hilos que las conectan. Un problema similar surge con el diseño de circuitos integrados a partir de disposiciones de módulos estándar e interconexiones entre ellos. Una generalización del problema de la colocación unidimensional de paquetes permite la colocación de los «paquetes», que tienen alto y ancho, en una región bidimensional, al tiempo que se minimiza la suma de las longitudes de los hilos entre los paquetes. Este problema también es aplicable en el diseño de circuitos integrados, entre otras áreas.

Hay ciertas transformaciones locales que se podrían utilizar para encontrar los óptimos locales para diversos casos del problema de la colocación unidimensional de paquetes. He aquí algunas:

1. Intercambiar los paquetes adyacentes p_i y p_{i+1} si el orden resultante es de costo menor. Sea $L(j)$ la suma de pesos de las aristas que se extienden hacia la izquierda de p_j , esto es, $\sum_{k=j+1}^n w(p_k, p_j)$. Igualmente, sea $R(j) = \sum_{k=j+1}^n w(p_k, p_j)$. Se obtienen mejoras si $L(i) - R(i) + R(i+1) - L(i+1) + 2w(p_i, p_{i+1})$ es negativa. Sería útil verificar esta fórmula calculando los costos antes y después del intercambio y tomar la diferencia.
2. Tomar un paquete p_i e insertarlo entre p_i y p_{j+1} para algunas i y j .
3. Intercambiar dos paquetes cualesquiera p_i y p_j .

Ejemplo 10.13. Supóngase que se tiene el grafo de la figura 10.21 para representar un caso de la colocación de paquetes. Se considerará sólo conjunto de transformaciones simples (1). Una colocación inicial, a, b, c, d, e , se muestra en la figura 10.28(a); tiene un costo de 97. Obsérvese que la función de costo pesa las aristas por su distancia, de modo que (a, e) contribuye $4 \times 7 = 28$ al costo de 97. Considérese el intercambio de d y e . Se tiene que $L(d) = 13$, $R(d) = 6$, $L(e) = 24$, y $R(e) = 0$. Así, $L(d) - R(d) + R(e) - L(e) + 2w(d, e) = -5$, y se puede intercambiar d y e para mejorar la colocación a (a, b, c, e, d) , con un costo de 92, como se muestra en la figura 10.28(b).

En la figura 10.28(b), se intercambian c y e con cierta ventaja, produciendo la figura 10.28(c), cuya colocación (a, b, e, c, d) tiene un costo 91. La figura 10.28(c) es localmente óptima para el conjunto de transformaciones (1). No es globalmente óptima; (a, c, e, d, b) tiene un costo de 84. $\square$

Como en el PAV, es difícil estimar exactamente el tiempo que lleva alcanzar un óptimo local. Sólo se observa que para el conjunto de transformaciones (1), hay sólo $n-1$ transformaciones a considerar. Más aún, si se calculan $L(i)$ y $R(i)$ una vez, tan sólo es necesario cambiarlas cuando p_i se intercambia con p_{i-1} o p_{i+1} . Además, calcular de nuevo es fácil. Si p_i y p_{i+1} se intercambian, por ejemplo, los nuevos $L(i)$ y

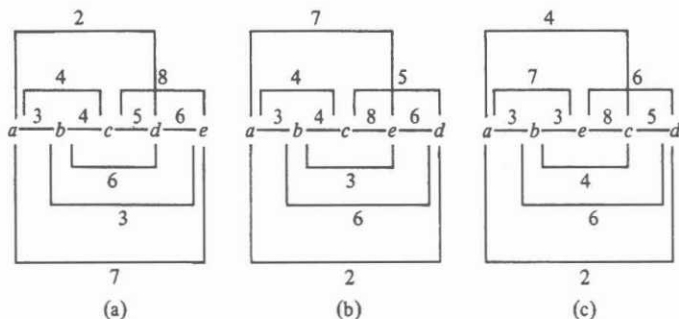


Fig. 10.28. Optimizaciones locales.

$R(i)$ son, respectivamente, $L(i+1) - w(p_i, p_{i+1})$ y $R(i+1) + w(p_i, p_{i+1})$. Así, un tiempo $O(n)$ es suficiente para probar una transformación de mejora y calcular de nuevo los $L(i)$ y $R(i)$. También se requiere un tiempo $O(n)$ para asignar valores iniciales a $L(i)$ y $R(i)$, si se emplea la recurrencia

$$L(1) = 0$$

$$L(i) = L(i-1) + w(p_{i-1}, p_i)$$

y una recurrencia similar para R .

En comparación, los conjuntos de transformaciones (2) y (3) tienen, cada uno, $O(n^2)$ miembros. Por tanto, llevará un tiempo $O(n^2)$ tan sólo confirmar que se tiene una solución localmente optimal. Sin embargo, como sucedió con el conjunto (1), no es posible acotar con exactitud el tiempo total requerido al efectuar una secuencia de mejoras, ya que cada mejora puede crear oportunidades adicionales de perfeccionamiento.

Ejercicios

- 10.1 ¿Cuántos movimientos realizan los algoritmos para desplazar n discos en el problema de las torres de Hanoi?
- *10.2 Pruébese que el algoritmo recursivo (dividir para vencer) de las torres de Hanoi y el algoritmo simple no recursivo descrito al principio de la sección 10.1 efectúan los mismos pasos.
- 10.3 Muéstrense las acciones del algoritmo de la multiplicación de enteros de dividir para vencer de la figura 10.3 al multiplicar 1011 por 1101.
- *10.4 Generalícese la programación del torneo de tenis de la sección 10.1 para torneos en los que el número de jugadores no sea una potencia de dos. *Suge-*

rencia. Si el número de jugadores n es impar, un jugador debe quedar sin jugar algún día, y el torneo necesitará n días para terminar, en vez de $n - 1$. No obstante, si hay dos grupos con un número impar de jugadores, los que obtienen el día sin jugar de cada grupo pueden jugar entre sí.

- 10.5 El número de combinaciones de m objetos de n , denotado por $\binom{n}{m}$, para $n \geq 1$ y $0 \leq m \leq n$, se puede definir recursivamente por

$$\binom{n}{m} = 1 \text{ si } m = 0 \text{ o } m = n$$

$$\binom{n}{m} = \binom{n-1}{m} + \binom{n-1}{m-1} \text{ si } 0 < m < n$$

- Proporciónese una función recursiva para calcular $\binom{n}{m}$.
- ¿Cuál es el tiempo de ejecución en el peor caso, como una función de n ?
- Obténgase un algoritmo de programación dinámica para calcular $\binom{n}{m}$. *Sugerencia:* El algoritmo construye una tabla conocida como triángulo de Pascal.
- ¿Cuál es el tiempo de ejecución de la respuesta a c) como una función de n ?

- 10.6 Otra forma de calcular el número de combinaciones de m objetos de n es mediante $(n)(n-1)(n-2) \dots (n-m+1)/(1)(2) \dots (m)$.

- ¿Cuál es el tiempo de ejecución en el peor caso de este algoritmo como una función de n ?
- *¿Es posible calcular la función de las «apuestas de la Serie Mundial de Béisbol» $P(i, j)$ de la sección 10.2 de una forma similar? ¿Con qué rapidez puede realizarse este cálculo?

- 10.7 a) Escribbase otra vez el cálculo de las apuestas de la figura 10.7 para tener en cuenta el hecho de que el primer equipo tiene una probabilidad p de ganar cualquier partido dado.

- Si los Dodgers han ganado un partido, y los Yankees, dos, pero los Dodgers tienen una probabilidad 0.6 de ganar cualquier juego, ¿quién es el ganador más probable de la Serie Mundial?

- 10.8 El cálculo de las apuestas de la figura 10.7 requiere un espacio $O(n^2)$. Reescribbase el algoritmo para usar sólo un espacio $O(n)$.

- *10.9 Pruébese que de la ecuación (10.4) resultan exactamente $\binom{i+j}{i}$ llamadas a P .

- 10.10 Encuéntrase una triangulación minimal para un octágono regular, suponiendo que las distancias son euclidianas.

- 10.11 El problema de división en párrafos, en una forma muy simple, puede formularse como sigue: se da una secuencia de palabras $p_1, p_2, \dots, p_k$ de longitudes $l_1, l_2, \dots, l_k$, que se desea agrupar en líneas de longitud L . Las palabras están separadas por espacios cuya amplitud ideal es b , pero los espa-

cios pueden reducirse o ampliarse si es necesario (pero sin solapamiento de palabras), de tal forma que una línea $p_i, p_{i+1} \dots p_j$ tenga exactamente longitud L . Sin embargo, la *multa* por reducción o ampliación es la magnitud de la cantidad total que los espacios se comprimen o amplían. Esto es, el costo de fijar la línea $p_i, p_{i+1} \dots p_j$ es $(j-i) |b'-b|$, donde b' el ancho real de los espacios, es $(L - l_i - l_{i+1} - \dots - l_j) / (j-i)$. No obstante, si $j = k$ (la última línea), el costo es cero, a menos que $b' < b$, ya que no es necesario ampliar la última línea. Plantee un algoritmo de programación dinámica para encontrar la separación de menor costo de $p_1, p_2, \dots, p_k$ en líneas de longitud L . *Sugerencia:* Para $i = k, k-1, \dots, 1$, calcúlese el menor costo de fijar $p_i, p_{i+1}, \dots, p_k$.

- 10.12 Supóngase que se dan n elementos $x_1, x_2, \dots, x_n$ relacionados por un orden lineal $x_1 < x_2 < \dots < x_n$, y que se desea disponer los m elementos dentro de un árbol binario de búsqueda. Supóngase que p_i es la probabilidad que se requiere de que una solicitud para encontrar un elemento se refiera a x_i . Entonces, para cualquier árbol binario de búsqueda, el costo promedio de una

búsqueda es $\sum_{i=1}^n p_i(d_i + 1)$, donde d_i es la profundidad del nodo que contiene a x_i . Dados los p_i y en el supuesto de que los x_i nunca cambian, es posible encontrar un árbol binario de búsqueda que reduzca el costo. Dése un algoritmo de programación dinámica para hacerlo. ¿Cuál es el tiempo de ejecución de ese algoritmo? *Sugerencia:* Calcúlese para todas las i y j el costo de búsqueda optimal entre todos los árboles que contienen sólo $x_i, x_{i+1}, \dots, x_{j+1}$, esto es, los j elementos comenzando desde x_i .

- **10.13 ¿Para qué valores de monedas produce una solución optimal el algoritmo ávido que calcula cambios de la sección 10.3?

- 10.14 a) Escribbase el algoritmo de triangulación recursivo analizado en la sección 10.2.

b) Muéstrase que el algoritmo recursivo produce exactamente 3^{n-4} llamadas a problemas no triviales cuando se inicia en un problema de tamaño $s \geq 4$.

- 10.15 Describbase un algoritmo ávido para

- a) el problema de la colocación unidimensional de paquetes, y
b) el problema de la separación en párrafos (Ejercicio 10.11). Proporciónese un ejemplo donde el algoritmo no produzca una respuesta optimal, o demuéstrese que tal ejemplo no existe.

- 10.16 Plantee una versión no recursiva del algoritmo del árbol de búsqueda de la figura 10.17.

- 10.17 Considérese un árbol de juego en el cual hay seis fichas y los jugadores 1 y 2 se turnan de una a tres fichas. El jugador que tome la última ficha *pierde* el juego.

- a) Dibújese el árbol de juego completo.
 - b) Si el árbol de juego completo se explorara mediante la técnica de poda alfa-beta y se analizaran primero los nodos que representan las configuraciones con el menor número de fichas, ¿qué nodos se podarían?
 - c) ¿Quién resulta vencedor si ambos jugadores hacen sus mejores jugadas?
- *10.18** Desarrollése un algoritmo de ramificación y acotamiento para el PAV, basado en la idea de comenzar un recorrido en el vértice 1 y de que en cada nivel la ramificación se basa en el nodo que sigue en el recorrido (en vez de en la elección de una arista en particular, como en la Fig. 10.22). ¿Qué estimación es correcta para la cota inferior de las configuraciones, que son listas de vértices $1, v_1, v_2, \dots$ que empiezan un recorrido? ¿Cómo se comporta el algoritmo en la figura 10.21, suponiendo que a es el vértice 1?
- *10.19** Un posible algoritmo de búsqueda local para el problema de la separación en párrafos es permitir transformaciones locales que pasen la primera palabra de una línea a la línea anterior o la última palabra de una línea a la línea siguiente. ¿Este algoritmo será localmente optimal, en el sentido de que toda solución localmente optimal es una solución globalmente optimal?
- 10.20** Si las transformaciones locales sólo consisten en opciones dobles, ¿hay algún recorrido localmente optimal en la figura 10.21 que no sea globalmente optimal?

Notas bibliográficas

Hay muchos ejemplos importantes de los algoritmos de clasificación del resultado por división, incluyendo la transformada rápida de Fourier $O(n \log n)$ de Cooley y Tukey [1965], el algoritmo de la multiplicación de enteros $O(n \log n \log \log n)$ de Schönhage y Strassen [1971], y el algoritmo de multiplicación de matrices $O(n^{2.81})$ de Strassen [1969]. El algoritmo de multiplicación de enteros $O(n^{1.59})$ es de Karatsuba y Ofman [1962]. Moenck y Borodin [1972] desarrollaron diferentes algoritmos eficientes del de divider para vencer para aritmética modular y evaluación e interpolación polinomiales.

La programación dinámica fue popularizada por Bellman [1957]. La aplicación de programación dinámica a la triangulación se debe a Fuchs, Kedem y Usselman [1977]. El ejercicio 10.11 es de Knuth [1981]. Knuth [1971] da una solución al problema de los árboles de búsqueda binaria optimales del ejercicio 10.12.

En Lin y Kernighan [1973] se describe una técnica heurística eficiente para el problema del agente viajero.

Véanse Aho, Hopcroft y Ullman [1974], y Garey y Johnson [1979] sobre un análisis de problemas NP-completos y otros problemas computacionalmente difíciles.

11

Estructuras de datos y algoritmos para almacenamiento externo

Este capítulo comienza considerando las diferencias en las formas de acceso entre la memoria principal y los dispositivos de almacenamiento externo como los discos. Después se presentan varios algoritmos para clasificación de archivos de datos almacenados en forma externa. Se concluye el capítulo con un análisis de estructuras de datos y algoritmos, como los archivos indizados y los árboles B, que son muy adecuados para el almacenamiento y recuperación de información en dispositivos de almacenamiento secundario.

11.1 Un modelo para cómputos con almacenamiento externo

En los algoritmos estudiados en capítulos anteriores, se ha supuesto que la cantidad de datos de entrada es lo bastante pequeña como para que quepan en la memoria al mismo tiempo. Pero, ¿qué sucede si se desea clasificar a todos los empleados de gobierno de acuerdo con su antigüedad, o almacenar toda la información de los impuestos de la nación? En problemas como éstos, la cantidad de datos por procesar supera la capacidad de la memoria principal. La mayor parte de los grandes sistemas de cómputo tienen dispositivos de almacenamiento externo conectados en línea, como discos o dispositivos de almacenamiento masivo, en los cuales se pueden almacenar cantidades muy grandes de datos. Sin embargo, esos dispositivos tienen características de acceso que difieren mucho de las de la memoria principal. Se han desarrollado diversas estructuras de datos y algoritmos para utilizar con más eficiencia esos dispositivos. Este capítulo comprende las estructuras de datos y algoritmos para clasificar y recuperar la información almacenada en memoria secundaria.

El lenguaje Pascal, y algunos otros, tienen el tipo de datos archivo, destinado a representar datos almacenados en memoria secundaria. Aunque el lenguaje utilizado no tenga este tipo de datos, es indudable que el sistema operativo debe manejar la noción de archivos en memoria secundaria. Tanto si se habla de los archivos de Pascal como de los manipulados directamente por el sistema operativo, se encuentran limitaciones a la forma en que puede accederse a los archivos. El sistema operativo divide la memoria secundaria en *bloques* de igual tamaño. El tamaño de los bloques varía entre los distintos sistemas operativos, pero de 512 a 4096 bytes es lo típico.

Se considera un archivo como si estuviera almacenado en una lista enlazada de bloques, aunque lo más común es que el sistema operativo utilice una disposición

tipo árbol, donde los bloques que contienen el archivo son hojas, y cada uno de los nodos interiores apunta a varios bloques del archivo. Si, por ejemplo, cuatro bytes son suficientes para contener la dirección de un bloque y los bloques tienen 4096 bytes de longitud, entonces un bloque raíz puede tener apuntadores hasta para 1024 bloques. Así, los archivos de hasta 1024 bloques, es decir, cerca de cuatro millones de bytes, pueden representarse con un bloque raíz y los bloques que contienen el archivo. Los archivos de hasta 2^{20} bloques, o 2^{32} bytes, pueden representarse con un bloque raíz que apunte a 1024 bloques en un nivel intermedio, cada uno de los cuales apunta a 1024 bloques_hoja que contienen una parte del archivo, y así sucesivamente.

La operación básica en archivos consiste en llevar un solo bloque a un *buffer* (almacenamiento temporal) de la memoria principal; un *buffer* no es más que un área reservada de memoria principal cuyo tamaño es idéntico al de un bloque. Un sistema operativo típico facilita la lectura de los bloques de acuerdo con el orden en que aparecen en la lista de bloques que conforman el archivo. Esto es, al principio se lee el primer bloque y se guarda en el *buffer* de ese archivo, después se reemplaza por el segundo, que queda escrito dentro del mismo *buffer*, y así sucesivamente.

Ahora se puede ver la razón de las reglas para la lectura de archivos en Pascal. Cada archivo está almacenado en una secuencia de bloques, con un número entero de registros en cada bloque. (Se puede desperdiciar espacio, puesto que se evita dividir un registro entre bloques.) El cursor de lectura siempre apunta a uno de los registros del bloque que se encuentra en el *buffer* en ese instante. Cuando ese cursor se deba mover a un registro que no esté en el *buffer*, es el momento de leer el siguiente bloque del archivo.

Igualmente, se puede considerar el proceso de escritura de archivos de Pascal como la creación de un archivo en un *buffer*. Cuando se «escriben» los registros en archivo, se colocan en el *buffer* de ese archivo, en la posición inmediata siguiente a los registros colocados previamente. Cuando el *buffer* no puede contener otro registro completo, se copia el *buffer* en un bloque disponible de memoria secundaria y ese bloque se agrega al final de la lista de bloques del archivo. Se considera ahora que el *buffer* está vacío, y que se pueden escribir más registros en él.

Costo de las operaciones con almacenamiento secundario

Dada la naturaleza de los dispositivos de almacenamiento secundario, como los discos, el tiempo para encontrar un bloque y leerlo a la memoria principal es muy grande, comparado con el tiempo que lleva procesar el dato. Por ejemplo, supóngase que se tiene un bloque de 1000 enteros dentro de un disco que gira a 1000 rpm. El tiempo que lleva colocar la cabeza sobre la pista que contenga este bloque (*tiempo de búsqueda*), más el tiempo consumido en esperar que el bloque quede bajo la cabeza (*tiempo de latencia*), puede ser de 100 milisegundos en promedio. El proceso de escritura de un bloque en un lugar particular dentro del almacenamiento secundario lleva una cantidad similar de tiempo. Sin embargo, la máquina puede efectuar 100 000 instrucciones en esos 100 milisegundos. Este tiempo es más que suficiente

para hacer un procesamiento simple a los mil enteros una vez que están en la memoria principal, como la suma de ellos o la ubicación del máximo; incluso puede ser suficiente para clasificación rápida de los enteros.

Cuando se evalúa el tiempo de ejecución de los algoritmos que operan sobre datos almacenados en forma de archivos, es de primordial importancia considerar el número de veces que se lee un bloque a la memoria principal o se escribe un bloque en la memoria secundaria; esa operación se denomina *acceso a bloques*. Se supone que el tamaño de los bloques lo fija el sistema operativo, así que no se puede hacer que un algoritmo se ejecute con mayor rapidez incrementando el tamaño del bloque, para reducir el número de accesos a los bloques. Como consecuencia, lo que se debe considerar para los algoritmos que emplean almacenamiento externo será el número de accesos a los bloques. Se inicia el estudio de algoritmos para almacenamiento externo con la clasificación externa.

11.2 Clasificación externa

La clasificación de datos organizados como archivos o de forma más general, la clasificación de datos almacenados en memoria secundaria, se conoce como clasificación «externa». Este estudio de clasificación externa parte del supuesto de que los datos están almacenados en un archivo de Pascal. Se presenta la forma en que un algoritmo de «clasificación por intercalación» puede ordenar un archivo de n registros en sólo $O(\log n)$ pasadas por el archivo; esta cifra es mucho mejor que los $O(n)$ pasos requeridos por los algoritmos estudiados en el capítulo 8. Después se estudia cómo la utilización de ciertas características del sistema operativo para controlar la lectura y escritura de bloques en tiempos adecuados puede acelerar la clasificación al reducir el tiempo en que el computador está ocioso, esperando la lectura o escritura de un bloque hacia o desde la memoria principal.

Clasificación por intercalación

La idea esencial en la clasificación por intercalación es organizar un archivo en *fragmentos* cada vez más grandes, esto es, secuencias de registros $r_1, \dots, r_k$, donde la clave de r_i no es mayor que la clave de r_{i+1} para $1 \leq i < k$. Se dice que un archivo $r_1, \dots, r_m$ de registros está *organizado en fragmentos de longitud k* si para toda $i \geq 0$ tal que $ki \leq m$, $r_{k(i-1)+1}, r_{k(i-1)+2}, \dots, r_{ki}$ es un fragmento de longitud k , y además si m no es divisible entre k , y $m = pk + q$, donde $q < k$, por lo que la secuencia de registros $r_{m-q+1}, r_{m-q+2}, \dots, r_m$, llamada *cola*, es un fragmento de longitud q . Por ejemplo, la secuencia de enteros mostrada en la figura 11.1 está organizada en fragmentos de longitud 3. Obsérvese que la cola es de longitud menor que 3, pero consta de registros ordenados, a saber, 5 y 12.

7	15	29	8	11	13	16	22	31	5	12
---	----	----	---	----	----	----	----	----	---	----

Fig. 11.1. Archivo con fragmentos de longitud tres.

El paso básico en una clasificación por intercalación en archivos es empezar con dos archivos, como f_1 y f_2 , organizados en fragmentos de longitud k . Suponiendo que

1. el número de fragmentos, incluyendo las colas, en f_1 y f_2 difiere a lo sumo en 1,
2. como máximo uno de f_1 y f_2 tiene cola, y
3. el único con cola tiene por lo menos tantos fragmentos como el otro.

Es un proceso sencillo leer un fragmento de f_1 y f_2 , combinar ambos y unir el fragmento resultante, de longitud $2k$, en alguno de los archivos g_1 y g_2 , que se han organizado en fragmentos de longitud $2k$. Alternando entre g_1 y g_2 , se hace que esos archivos no sólo se organicen en fragmentos de longitud $2k$, sino que satisfagan (1), (2) y (3). Para ver que (2) y (3) se satisfacen, es útil observar que la cola entre los fragmentos de f_1 y f_2 queda combinada en (o quizá ya está) el último fragmento creado.

Se empieza por dividir los n registros en dos archivos f_1 y f_2 , lo más uniformemente posible. Cualquier archivo se puede considerar organizado en fragmentos de longitud 1. Después, se combinan en fragmentos de longitud 1 y se distribuyen en los archivos g_1 y g_2 , organizados en fragmentos de longitud 2. Se vacían f_1 y f_2 , y se combinan g_1 y g_2 en f_1 y f_2 , ahora estarán organizados en fragmentos de longitud 4. Después, se combinan f_1 y f_2 para crear g_1 y g_2 organizados en fragmentos de longitud 8, y así sucesivamente.

Después de i pasadas de esta naturaleza, se tendrán dos archivos constituidos por fragmentos de longitud 2^i . Si $2^i \geq n$, uno de los dos archivos estará vacío y el otro tendrá un solo fragmento de longitud n , esto es, estará clasificado. Como $2^i \geq n$ cuando $i \geq \log n$, $\lceil \log n \rceil$ pasadas son suficientes. Cada pasada requiere la lectura de dos archivos y la escritura de otros dos, longitud aproximada $n/2$. El número total de bloques leídos o escritos en cada paso es aproximadamente $2n/b$, donde b es el número de registros que caben en un bloque. De este modo, el número de bloques leídos y escritos por el proceso de clasificación completo es $O((n \log n)/b)$ o, de otra forma, la cantidad de lecturas y escrituras es casi la misma que la requerida para hacer $O(\log n)$ pasadas por los datos almacenados en un solo archivo. Esta cifra supone una importante mejora de las $O(n)$ pasadas requeridas por muchos de los algoritmos de clasificación tratados en el capítulo 8.

La figura 11.2 muestra el proceso de combinación en Pascal. Se leen dos archivos organizados en fragmentos de longitud k y se escriben dos archivos organizados en fragmentos de longitud $2k$. Como ejercicio, se deja la especificación de un algoritmo que siga las ideas anteriores y que utilice el procedimiento *combina* de la figura 11.2 $\log n$ veces para clasificar un archivo de n registros.

```

procedure combina (  $k$ : integer; { longitud del fragmento de entrada }
     $f_1, f_2, g_1, g_2$ : file of tipo_registro );
var
    conmuta_salida: boolean; { indica si la escritura es en  $g_1$  (verdadero) o en
         $g_2$  (falso) }
    ganador: integer; { elige el archivo con la menor clave del registro actual }
    usado: array [1..2] of integer; { usado[ $j$ ] indica cuántos registros se han
        leído hasta ahora en el fragmento actual del archivo  $f_j$  }
    fin: array [1..2] of boolean; { fin[ $j$ ] es verdadero si ha terminado el

```

```

    fragmento de  $f_1$  - se han leído  $k$  registros o se ha alcanzado el fin del
    archivo de  $f_j$  }
    actual: array [1..2] of tipo_registro; { los registros actuales de ambos
    archivos }
procedure toma_registro ( i: integer ); { avanza el archivo  $f_i$ , pero no más allá
    del final del archivo o del fragmento. Fija fin[i] si se encuentra el fin del
    archivo o del fragmento }
begin
    usado[i] := usado[i] + 1;
    if (usado[i] = k) or
        (i = 1) and eof( $f_1$ ) or
        (i = 2) and eof( $f_2$ ) then fin[i] := true
    else if i = 1 then read( $f_1$ , actual[1])
    else read( $f_2$ , actual[2])
    end; { toma_registro }
begin { combina }
    conmuta_salida := true; { los primeros fragmentos intercalados van hacia  $g_1$  }
    rewrite( $g_1$ ); rewrite( $g_2$ );
    reset( $f_1$ ); reset( $f_2$ );
    while not eof( $f_1$ ) or not eof( $f_2$ ) do begin { intercala dos archivos }
        { asigna valor inicial }
        usado[1] := 0; usado[2] := 0;
        fin[1] := false; fin[2] := false;
        toma_registro(1); toma_registro(2);
        while not fin[1] or not fin[2] do begin { intercala dos fragmentos }
            { elige ganador }
            if fin[1] then ganador := 2
                {  $f_2$  gana por "omisión"; el fragmento de  $f_1$  se terminó }
            else if fin[2] then ganador := 1
                {  $f_1$  gana por omisión }
            else { ningún fragmento se ha terminado }
                if actual[1].clave < actual[2].clave then ganador := 1
                else ganador := 2;
            { escribe el registro del ganador }
            if conmuta_salida then write( $g_1$ , actual[ganador])
            else write( $g_2$ , actual[ganador]);
            { avanza el archivo ganador }
            toma_registro(ganador)
        end;
        { se ha terminado la intercalación de dos fragmentos; se conmuta el
        archivo de salida y se repite }
        conmuta_salida := not conmuta_salida
    end
end; { combina }

```

Fig. 11.2. Procedimiento *combina*.

Obsérvese que el procedimiento *combina* de la figura 11.2 nunca requiere un fragmento completo en memoria; lee y escribe un registro de cada vez. No se desea almacenar los fragmentos completos que se encuentran en memoria principal que obliguen a tener dos archivos de entrada. En otro caso, se podrían leer dos fragmentos al mismo tiempo desde un solo archivo.

Ejemplo 11.1. Considérese la lista de 23 números que se muestra dividida en dos archivos en la figura 11.3(a). Se empieza por combinar fragmentos de longitud 1 para crear los dos archivos de la figura 11.3(b). Por ejemplo, los primeros fragmentos de longitud uno son 28 y 31, y se combinan tomando primero 28 y luego 31. Los dos fragmentos siguientes de longitud uno, 3 y 5, se combinan para formar uno de longitud dos, que se coloca en el segundo archivo de la figura 11.3(b). Los fragmentos se separan en la figura 11.3(b) por medio de líneas verticales que no son parte del archivo. Obsérvese que el segundo archivo de la figura 11.3(b) tiene una cola de longitud uno, el registro 22, mientras que el primer archivo no tiene cola.

Se pasa, de la figura 11.3(b) a la figura 11.3(c) combinando los fragmentos de longitud dos. Por ejemplo, los fragmentos 28, 31 y 3, 5, se combinan para formar 3, 5, 28, 31 de la figura 11.3(c). Al alcanzar los fragmentos de longitud 16 de la figura 11.3(e), un archivo tiene un fragmento completo y el otro sólo tiene una cola, de longitud 7. En la última etapa, donde los archivos están en apariencia organizados como fragmentos de longitud 32, sucede que se tiene un archivo formado sólo por una cola, de longitud 23, y el segundo se encuentra vacío. El fragmento sencillo de longitud 23 es, por supuesto, la clasificación que se buscaba. □

Aceleración de la clasificación por intercalación

Por medio de un ejemplo simple se ha mostrado el proceso de la clasificación por intercalación partiendo de fragmentos de longitud 1. Se aprovechará más el tiempo si se empieza con un paso que, para una k apropiada, lea grupos de k registros dentro de la memoria principal, los ordene, por ejemplo, con clasificación rápida, y los devuelva a la memoria secundaria como un fragmento de longitud k .

Por ejemplo, un millón de registros, necesitarían 20 pasadas por los datos para hacer la clasificación empezando con fragmentos de longitud 1. Sin embargo, si se pueden tener 10 000 registros a la vez en la memoria principal, también se pueden leer 100 grupos de 10 000 registros en una pasada, clasificar cada grupo, y dejar 100 fragmentos de longitud 10 000 distribuidos uniformemente entre dos archivos. Siete pasadas de intercalación más culminarán con un archivo organizado como un fragmento de longitud $10\,000 \times 2^7 = 1\,280\,000$, lo que es mayor que un millón y significa que los datos están clasificados.

Minimización del tiempo transcurrido

28	3	93	10	54	65	30	90	10	69	8	22
31	5	96	40	85	9	39	13	8	77	10	

(a) archivos iniciales

28	31		93	96		54	85		30	39		8	10		8	10
3	5		10	40		9	65		13	90		69	77		22	

(b) organizados en fragmentos de longitud 2

3	5	28	31		9	54	65	85		8	10	69	77
10	40	93	06		13	30	39	90		8	10	22	

(c) organizados en fragmentos de longitud 4

3	5	10	28	31	40	93	96		8	8	10	10	22	69	77
9	13	30	39	54	65	85	90								

(d) organizados en fragmentos de longitud 8

3	5	9	10	13	28	30	31	39	40	54	65	85	90	93	96
8	8	10	10	22	69	77									

(e) organizados en fragmentos de longitud 16

3	5	8	8	9	10	10	10	13	22	28	30	31	39	40	54	65	69	77	85	90	93	96
---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

(f) organizados en fragmentos de longitud 32

Fig. 11.3. Clasificación por intercalación de una lista.

chivo que se está leyendo, tal como sucede durante el proceso de clasificación por intercalación. Sin embargo, el hecho es que el tiempo transcurrido por dicho proceso es mayor que el tiempo empleado en el cálculo efectuado con los datos presentes en la memoria principal. Si se clasifican archivos realmente grandes, donde la operación lleva horas, el tiempo transcurrido resulta un factor importante, aun si no se paga, y debe buscarse la forma de que el proceso de la clasificación por intercalación lleve un tiempo total mínimo.

Como se ha mencionado, es común que el tiempo para leer datos de disco o cin-

ta sea mayor que el tiempo consumido en hacer cálculos simples con esos datos, como la intercalación. Por tanto, es de esperar que si sólo hay un canal dedicado a la transferencia de datos entre la memoria principal y la memoria auxiliar, este canal formará un embotellamiento, el canal de datos estará ocupado todo el tiempo, y el tiempo total transcurrido se igualará con el tiempo utilizado en el movimiento de los datos. Esto es, todos los cálculos se efectuarán en cuanto los datos se encuentren disponibles, mientras se leen o escriben datos adicionales.

Aun en este ambiente simple, debe tenerse cuidado para asegurar la terminación en una cantidad mínima de tiempo. Para ver qué puede fallar, supóngase que se lee, cada vez un bloque, de dos archivos de entrada f_1 y f_2 , en forma alterna. Los archivos están organizados en fragmentos de longitud bastante mayor que el tamaño de un bloque, así que para intercalar dos fragmentos es necesario leer muchos bloques de cada archivo. Sin embargo, supóngase que todos los registros contenidos en el fragmento del archivo f_1 preceden a todos los registros del archivo f_2 . Entonces, como se leen bloques en forma alterna, todos los bloques de f_2 tienen que permanecer en memoria. Es posible que no haya espacio para conservar todos esos bloques en la memoria principal, y aun si lo hubiera, es necesario, después de leer todos los bloques del fragmento, esperar mientras se copia y escribe todo el fragmento que proviene de f_2 .

Para evitar esos problemas, se consideran las claves de los últimos registros de los últimos bloques leídos de f_1 y f_2 , por ejemplo k_1 y k_2 , respectivamente. Si algún fragmento se agota, es obvio que se lee la siguiente de otro archivo. Sin embargo, si un fragmento no se agota, se continúa leyendo un bloque de f_1 si $k_1 < k_2$, y de f_2 , en caso contrario. Esto es, debe determinarse cuál de los dos fragmentos tendrá primero todos sus registros seleccionados en ese momento en la memoria principal, y se rellenan primero registros de ese fragmento. Si la selección de registros se efectúa más rápido que la lectura, se sabe que al haber leído el último bloque de los dos fragmentos, no puede haber más que dos bloques llenos de registros sin intercalar; los registros pueden estar distribuidos en tres bloques, pero no en más.

Intercalación múltiple

Si la lectura y escritura entre las memorias principal y secundaria es el «embotellamiento», puede ahorrarse tiempo si se tiene más de un canal de datos. Supóngase que se tienen $2m$ unidades de disco, cada una con su propio canal de comunicación. Se podrían colocar m archivos, $f_1, f_2, \dots, f_m$ en m de las unidades de disco, organizados como fragmentos de longitud k . Entonces, se pueden leer m fragmentos, uno de cada archivo, y combinarlos en un fragmento de longitud mk , el cual se coloca en alguno de los m archivos de salida, $g_1, g_2, \dots, g_m$, cada uno tomando un fragmento a la vez.

El proceso de intercalación en memoria principal puede llevarse a cabo en $O(\log m)$ pasos por registro si se organizan los m registros candidatos, esto es, los registros más pequeños de cada archivo que hasta ese momento no se habían seleccionado, en un árbol parcialmente ordenado u otra estructura de datos que maneje

las operaciones en colas de prioridad INSERTA y SUPRIME-MIN en tiempo logarítmico. Para seleccionar el registro con la clave más pequeña de la cola de prioridad, se realiza SUPRIME-MIN, y después INSERTA, en la cola de prioridad del siguiente registro del archivo del ganador, como reemplazo del registro seleccionado.

Si hay n registros, y la longitud de los fragmentos se multiplica por m en cada paso, después de i pasadas los fragmentos serán de longitud m^i . Si $m^i \geq n$, esto es, después de $i = \log_m n$ pasadas, la lista completa estará clasificada. Como $\log_m n = \log_2 n / \log_2 m$, se ahorra en un factor de $\log_2 m$ el número de veces que se lee cada registro. Más aún, si m es el número de unidades de disco utilizadas para los archivos de entrada, y m son usadas para la salida, es posible procesar datos m veces tan rápido como si existiera sólo una unidad de disco para la entrada y una para la salida, o $2m$ veces tan rápido como si los archivos de entrada y salida estuvieran almacenados en una unidad de disco. Lamentablemente, incrementar m en forma indefinida no acelera el procesamiento en un factor de $\log m$. La razón de esto es que para una m suficientemente grande, el tiempo requerido por la intercalación en memoria principal, que en realidad se incrementa como $\log m$, será superior al de lectura o escritura de datos. En este punto, si hay incrementos posteriores en m , aumentarán el tiempo transcurrido, ya que el cálculo en memoria principal se convertirá en el embotellamiento.

Clasificación en varias fases

Es posible realizar una clasificación por intercalación de m -caminos con sólo $m + 1$ archivos, como alternativa a la estrategia descrita antes, que emplea $2m$ archivos. Se efectúa una secuencia de pasadas al intercalar fragmentos de m de los archivos en otros más largos en el archivo restante. Es necesario tener en cuenta los siguientes:

1. En una pasada, cuando los fragmentos de cada uno de los m archivos se intercalan en fragmentos del $(m + 1)$ -ésimo archivo, no es necesario usar todos los fragmentos en cada uno de los m archivos de entrada. Antes bien, cada archivo, cuando es de salida, se ocupa con fragmentos de cierta longitud. Se usan algunos de esos fragmentos para ayudar a llenar cada uno de los otros m archivos cuando les llegue el turno de ser archivos de salida.
2. Cada paso produce archivos de longitud diferente. Puesto que cada uno de los archivos cargados con fragmentos en las m pasadas previas contribuye a los fragmentos del paso actual, la longitud en una pasada es la suma de las longitudes de los fragmentos producidos en las m pasadas previas. (Si se han dado menos de m pasadas, habrá que considerar todos los anteriores como si produjeran fragmentos de longitud 1.)

Este proceso de clasificación por intercalación se conoce como *clasificación en varias fases* (*polyphase sorting*). El cálculo exacto de los números de pasadas necesarias como una función de m y n (el número de registros), y el cálculo de la distribución inicial de los fragmentos en n archivos se dejan como ejercicio. Sin embargo, aquí se dará un ejemplo para dar idea del caso general.

Ejemplo 11.2. Si $m = 2$, se comienza con dos archivos f_1 y f_2 , organizados en fragmentos de longitud 1. Los registros de f_1 y f_2 se intercalan para hacer fragmentos de longitud 2 en un tercer archivo, f_3 . Sólo se intercalan suficientes fragmentos para vaciar f_1 . Después, se intercalan los restantes de longitud 1 de f_2 con un número igual de fragmentos de longitud 2 de f_3 . Los resultantes, de longitud 3, quedarán colocados en f_1 . Después, deben intercalarse los de longitud 2 de f_3 con los de longitud 3 de f_1 . Esos fragmentos, de longitud 5, se colocan en f_2 , que quedó vacío en la pasada anterior.

La secuencia de longitudes de los fragmentos 1, 1, 2, 3, 5, 8, 13, 21, ..., es la sucesión de *Fibonacci*. Esta secuencia se genera por medio de la relación de recurrencia $F_0 = F_1 = 1$, y $F_i = F_{i-1} + F_{i-2}$, para $i \geq 2$. Obsérvese que la razón entre los números de Fibonacci consecutivos F_{i+1}/F_i se acerca a la «razón dorada» $(\sqrt{5} + 1)/2 = 1.618...$ conforme i crece.

De aquí se deduce que para que el proceso continúe hasta que la lista quede clasificada, los números iniciales de registros en f_1 y f_2 deben ser dos números consecutivos de Fibonacci. Por ejemplo, la figura 11.4 muestra qué sucede al empezar con $n = 34$ registros (34 es el número de Fibonacci F_8), distribuidos 13 en f_1 y 21 en f_2 . (13 y 21 son el sexto y el séptimo números de Fibonacci, así que la razón F_7/F_6 es muy cercana a 1.618; de hecho, es 1.615.) El estado de un archivo se representa en la figura 11.4 como $a(b)$, lo cual significa que tiene a fragmentos de longitud b . □

después de la pasada	f_1	f_2	f_3
inicial	13(1)	21(1)	vacío
1	vacío	8(1)	13(2)
2	8(3)	vacío	5(2)
3	3(3)	5(5)	vacío
4	vacío	2(5)	3(8)
5	2(13)	vacío	1(8)
6	1(13)	1(21)	vacío
7	vacío	vacío	1(34)

Fig. 11.4. Ejemplo de clasificación en varias fases.

Un caso en el que la velocidad de entrada/salida no es un cuello de botella

Cuando la lectura de archivos es el cuello de botella, el siguiente bloque debe escogerse con sumo cuidado. Como ya se dijo, la situación a evitar es aquella en la que se tienen que almacenar varios bloques de un fragmento, debido a que ese fragmento tenía registros con claves grandes, los cuales serán escogidos después de la mayor parte o de todos los registros del otro fragmento. El truco para evitar este problema consiste en determinar con rapidez qué fragmento agotará primero sus registros en memoria principal, comparando los últimos de esos registros de cada archivo.

Cuando el tiempo requerido para leer datos en la memoria principal es comparable o menor que el tiempo requerido para procesar los datos, se vuelve aún más crítica la selección del archivo de entrada desde el cual leer un bloque con cuidado, ya que no hay esperanza de construir una reserva de registros dentro de la memoria principal en caso de que el proceso de intercalación empiece repentinamente al tomar más registros de un fragmento que del otro. El «truco» mencionado antes resulta útil en diversas situaciones, como se verá a continuación.

Considérese el caso donde la intercalación es el cuello de botella, y no la lectura o la escritura, por dos razones.

1. Como se ha visto, al tener disponibles varias unidades de disco o cinta, es posible acelerar la entrada/salida lo suficiente para que el tiempo requerido por la intercalación supere al tiempo de entrada o al de salida.
2. Los canales de alta velocidad pueden estar pronto disponibles en el mercado.

Por tanto, se considerará un modelo simple del problema que se puede presentar cuando la intercalación es el cuello de botella en una clasificación realizada con datos almacenados en la memoria secundaria. Específicamente, se supone que

- a) Se intercalan fragmentos mucho más grandes que los bloques.
- b) Hay dos archivos de entrada y dos de salida. Los archivos de entrada están almacenados en un disco (o algún otro dispositivo conectado a la memoria principal a través de un solo canal) y los archivos de salida están en otra unidad similar con un solo canal.
- c) Los tiempos para
 - I) leer un bloque
 - II) escribir un bloque, y
 - III) seleccionar suficientes registros con las claves más pequeñas entre los dos fragmentos que se encuentran en ese momento en memoria principal, para llenar un bloque.

son todos iguales.

En dichos supuestos, se considera una clase de estrategias de intercalación donde varios buffers de entrada (espacios para contener un bloque) se ubican en la memoria principal. En todo momento, alguno de esos buffers contendrá los registros no seleccionados de los dos fragmentos de entrada, y uno de ellos estará en el proceso de ser leído de uno de los archivos de entrada. Los otros dos contendrán la salida, es decir, los registros seleccionados en el orden de intercalación adecuado. En todo momento, uno de esos buffers se encontrará en el proceso de escritura en alguno de los archivos de salida y el otro se estará llenando con registros seleccionados de los buffers de entrada.

Una *transferencia* consiste en hacer lo siguiente (quizá todo al mismo tiempo):

1. la lectura de un bloque de entrada en un buffer de entrada.
2. el llenado de uno de los buffers de salida con los registros seleccionados, es decir, los registros con las claves más pequeñas, entre todos los que se tengan en ese momento en el buffer de entrada.

3. la escritura del otro buffer de salida en uno de los dos archivos de salida que se esté generando.

Por las suposiciones, (1), (2) y (3) requieren el mismo tiempo. Para obtener el máximo de eficiencia, se deben efectuar en paralelo. Se puede hacer esto, a menos que (2), la selección de registros con las claves más pequeñas, incluya algunos de los registros que se están leyendo en ese momento †. Así, se debe buscar una estrategia para seleccionar los buffers que se van a leer, de modo que al principio de cada transferencia los b registros no seleccionados con las claves menores ya se encuentren listos en los buffers de entrada, donde b es el número de registros que llenan un bloque o buffer.

Las condiciones en las cuales la intercalación puede efectuarse en paralelo con la lectura son simples. Sean k_1 y k_2 las claves más grandes de todos los registros no seleccionados en memoria principal del primero y segundo fragmentos, respectivamente. Entonces, en la memoria principal deben encontrarse por lo menos b registros no seleccionados cuyas claves no excedan de $\min(k_1, k_2)$. En primer lugar se mostrará cómo hacer la intercalación con seis buffers, tres para cada archivo, y después se mostrará que es suficiente con cuatro buffers si se reparten entre los dos archivos.

Esquema de seis buffers de entrada

Este primer esquema se representa en el dibujo de la figura 11.5. Los dos buffers de salida no se muestran; existen tres buffers para cada archivo; cada uno tiene capacidad para b registros. El área sombreada representa los registros disponibles, y las claves están en orden ascendente en el sentido de las manecillas del reloj. Siempre, el número total de registros no seleccionados es $4b$ (a menos que el número de registros que permanecen en los fragmentos que se encuentran intercalando sea menor). Inicialmente, se leen los dos primeros bloques de cada fragmento en los buffers ††. Como siempre hay $4b$ registros disponibles, y a lo sumo $3b$ pueden proceder de un archivo, se sabe que hay por lo menos b registros que vienen de cada archivo. Si k_1 y k_2 son las claves más grandes disponibles de los dos fragmentos, debe haber b registros con las claves iguales o menores que k_1 y b registros con las claves menores o iguales que k_2 . Así, hay b registros con claves iguales o menores que $\min(k_1, k_2)$.

† Es tentador suponer que si (1) y (2) requieren el mismo tiempo, entonces la selección no podría nunca coincidir con la lectura; si el bloque completo no se hubiera leído aún, podrían seleccionarse de entre los primeros registros del bloque aquellos que tuvieran las claves menores. Sin embargo, por su naturaleza, en las lecturas de disco transcurre un periodo largo antes de que se encuentre el bloque y pueda realizarse la lectura. Por tanto, la única suposición segura es que nada del bloque que se está leyendo en una transferencia está disponible en ese momento para la selección.

†† Si éstos no son los primeros fragmentos tomados de cada archivo, entonces este paso inicial únicamente puede hacerse después de leer los anteriores fragmentos y sus últimos $4b$ registros se estén intercalando.

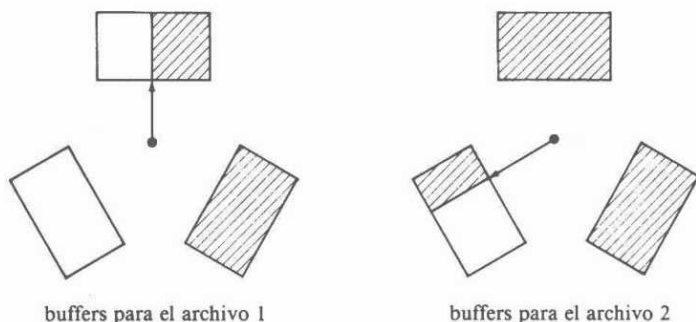


Fig. 11.5. Intercalación con seis buffers de entrada.

La pregunta sobre qué archivo leer a continuación es trivial. Por lo general, dado que dos buffers estarán llenos parcialmente, como en la figura 11.5, habrá sólo un buffer vacío y deberá llenarse. Si sucede, como cuando se está empezando, que cada fragmento tiene dos buffers completamente llenos y uno vacío, se toma cualquiera de los que se encuentran vacíos. Obsérvese que la demostración de que no es posible agotar un fragmento [existen b registros con claves iguales o menores que $\min(k_1, k_2)$] dependía sólo del hecho de que estuvieron presentes $4b$ registros.

Además, las flechas de la figura 11.5 representan apuntadores a los primeros registros disponibles (cuyas claves son menores) en ambos fragmentos. En Pascal, se representaría ese apuntador con dos enteros. El primero, en el intervalo $1..3$, representa el buffer apuntado, y el segundo, en el intervalo $1..b$, representa el registro dentro del buffer. En forma alternativa, es posible dejar que los buffers sean el primero, el medio y el último tercios de un arreglo y usar un entero en el intervalo $1..3b$. En otros lenguajes, donde los apuntadores pueden apuntar hacia elementos de arreglos, puede preferirse un apuntador de tipo $\uparrow$ tipo_registro.

Esquema de cuatro buffers

La figura 11.6 sugiere un esquema con cuatro buffers. En el principio de cada transferencia, están disponibles $2b$ registros. Dos de los buffers de entrada están asignados a uno de los archivos; B_1 y B_2 de la figura 11.6 están asignados al archivo uno. Uno de estos buffers estará parcialmente lleno (vacío en el caso extremo), y el otro, lleno. El cuarto buffer no está comprometido, y se llenará desde uno de los archivos durante la transferencia.

Se mantendrá, por supuesto, la propiedad que permite intercalar en paralelo con la lectura; al menos b registros de la figura 11.6 deben tener claves menores o iguales que $\min(k_1, k_2)$, donde k_1 y k_2 son las claves de los últimos registros disponibles en los dos archivos, como se indica en la figura 11.6. Se denomina *segura* a la configuración que cumple esa propiedad. Al principio, se lee un bloque de cada frag-